جامعة الإسكندرية
ALEXANDRIA UNIVERSITY
كلية الحاسبات وعلوم البيانات

Graduation Project Documentation

News Analysis System: A chatbot For News Analysis and Classification

SUBMITTED BY:

Yomna Muhammed Alaa Eskandar

Shahd Ahmed Taher

Manar Muhammed Saad

Engy Muhammed Hassan

Noureen Muhamed Ali

Under the Supervision of:

Dr. Christine Basta

Faculty of computer and Data Science,

Alexandria university

2023-2024

APPROVAL PAGE

Graduation Project Documentation

News Analysis System: A chatbot For News Analysis and Classification

Presented by:

Yomna Muhammed Alaa Eskandar

Shahd Ahmed Taher

Manar Muhammed Saad

Engy Muhammed Hassan

Noureen Muhamed Ali

Advisor:

Dr. Christine Basta

Alexandria university

2023-2024

ACKNOWLEDGEMENTS

This project was successfully completed thanks to the engagement and contributions of many people, especially those who took the time to offer their well-considered advice and suggestions.

We would like to express our deepest gratitude to our project advisor, Dr. Christine Basta, for her invaluable guidance, unwavering support, and insightful feedback throughout the development of this project. This project would not have been completed if it wasn't for her guidance and care for us and for the project.

We also would like to thank Eng. Hadeer Adel for her help and guidance in our project.

We also would like to thank ourselves for not giving up and for overcoming a lot of challenges.

Moreover, we would like to thank all our professors for the quality of teaching they gave us during our studies. A Special thanks to our family and parents for supporting and believing in our work throughout the learning journey.

DECLARATION

We hereby certify that this material, which we now submit for assessment on the program of study leading to the award of Bachelor of Science from Alexandria University is entirely our own work, that we have exercised reasonable care to ensure that the work is original and has not been taken from the work of others save and to the extent that such work has been cited and acknowledged within the text of our work.

Date: 2023-2024

Abstract

News can have a huge impact on our society, influencing our attitudes toward a variety of topics, beginning with changing our perspectives on foreign countries and potentially influencing the overall health of the economy. Therefore, news networks can also play a significant role in instilling good views in individuals, resulting in cooperation with the government and national growth.

The project endeavors to revolutionize news consumption in the digital era, addressing several pervasive challenges. Introducing a cutting-edge News Classification Chatbot, the project employs advanced technologies, including BeautifulSoup for web scraping, NLP for text summarization, generation and classification, and topic modeling for categorization. By amalgamating these techniques, the chatbot not only condenses lengthy articles into concise summaries but also classifies them into relevant topics, offering users an efficient and tailored news experience.

The heart of the project lies in its innovative NLP script. A seamless integration of multiple other components using Flask creates a powerful backend system. The culmination of this effort is an engaging chatbot interface developed with Dialog Flow, providing users with an interactive platform for accessing summarized and categorized news updates. The project's impact extends beyond mere technological advancement; its deployment on popular social media platforms positions the News Classification Chatbot as a game-changer, empowering users to navigate the digital news landscape with confidence, discernment, and a touch of technological flair.

Furthermore, the project extends its capabilities by incorporating the news summarization with headline generation. This feature allows users to receive concise and informative summaries of news articles, enhancing accessibility and understanding.

Table of Contents

Table of Contents 6
Chapter 1: Introduction 11
1.1 Problem Statement 12
1.2 Goal 13
1.3 Objectives 13
1.4 Aim of work 14
Chapter 2: Background 16
2.1 Introduction 16
2.2 Background 16
2.3 Deep Learning 16
2.4 Natural Language Processing 17
2.5 Language models 18
2.6 Transformers 19
2.7 Large Language Model 20
2.8 Llama 2 20
2.9 BERT 22
2.10 BERTopic 23
Chapter 3: Methodology and Results 26
3.1 General Description of the Project 26
3.2 Tasks and models 27
3.2.1 Headline Generation 27
3.2.2 Breaking News Classification 32
3.2.3 Topic Modeling 41
3.2.3 Text summarization 49
3.3 Environment 53
3.4 Chatbot 54
3.4.1 Dialogflow 55
3.4.2 The Fulfillment Logic 58
3.4.3 The Flask Server 58
3.4.4 The Webhook 58
3.4.5 Ngrok 59

3.4.6 Connections 59
3.4.7 The Web Scraper 60
4.4.8 Knowledge Base 61
Chapter 4: Conclusion 63
4.1 Future work 63
4.2 Challenges 64
4.3 Conclusion 64
Appendix A 66
A demo of the Chatbot 66
References 70

LIST OF FIGURES

Figure 1:Our project Logo 12
Figure 2: Transformer Architecture 19
Figure 3: Shows the project components 27
Figure 4:shows lama 2 system prompts 30
Figure 5:shows Lables percentage in Breaking and non_breaking classification data 34
Figure 6: Shows a word cloud 36
Figure 7: shows the graph of accurcy and lose of Breaking and non_breaking classification model 39
Figure 8:shows confusion matrix of Breaking and non_breaking classification model 40
Figure 9 Shows the 10 top words of topic 7 48
Figure 10:shows the average token count for the articles and the highlights 50
Figure 11:The data was split into 3 datasets 50
Figure 12 Shows an overview of the connections 55
Figure 13:shows dialogflow usage 56
Figure 14:shows intents used in project 57
Figure 15 showing the logo of the chatbot 66
Figure 16 showing a scenario where a user greets the chatbot 66
Figure 17 one article and its report 67
Figure 18 A question and the answer 67
Figure 19 shows the user requesting news 68
Figure 20: shows system prompt in topic modeling 68
Figure 21: shows User prompt in topic modeling 69
Figure 22: shows main prompt in topic modeling 69

LIST OF TABLES

Table 1:shows data used in Headline generation 28

Table 2:shows data used in Breaking and non_breaking classification 33

Table 3: shows topic modeling data used, bertkey and clusters 47

Table 4: shows the dataset used in text summarization 50

Table 5:Shows Environment Software 53

Table 6:Shows Environment hardware 54

Table 7:shows CSV file used in Knowledge base 62

LIST OF EQUATIONS

Equation 1 Accuracy Formula 32
Equation 2 Precision Formula 51
Equation 3 Recall formula 52
Equation 4 F1 score formula 52

Chapter 1: Introduction

In an era defined by the rapid dissemination of information, social media platforms and renowned news websites have emerged as powerful channels for news consumption. The ease of access and the ability to share news articles and updates with a vast audience have revolutionized the way we stay informed. However, the very advantages that make these platforms effective news sources also present significant challenges. The sheer volume of news, combined with its unorganized and often unreliable nature, can overwhelm users and hinder their ability to consume and comprehend information effectively.

In the realm of news consumption, users face several significant problems and difficulties which the News Analysis system aims to solve such as information overload, organization of information and of course the constant concern of reliability and credibility.

A large majority of U.S. adults (86%) say they often or sometimes get news from a smartphone, computer or tablet, including 56% who say they do so often. The portion that gets news from digital devices continues to outpace those who get news from television. When asked which of these platforms they *prefer* to get news on, nearly six-in-ten Americans say they prefer a digital device (58%), more than say they prefer TV (27%). Even fewer Americans prefer radio (6%) or print (5%).

Today, news websites, apps and search engines are the digital pathways most Americans get news from at least sometimes. Among digital platforms, news websites or apps are also the most preferred source for news: A quarter of U.S. adults prefer to get their news this way, compared with 15% who prefer search, 12% who prefer social media and 6% who say they prefer podcasts. Moreover, this shows how significant a news analysis system is and how much it is needed as it holds significant importance in addressing the contemporary challenges associated with news consumption in the digital age which has now become the primary source of news consumption.

1.1 Problem Statement

In the current digital landscape, the proliferation of news content has reached unprecedented levels, resulting in an overwhelming influx of daily information. This substantial increase in news volume presents a significant challenge in discerning and prioritizing critical news items. Consequently, individuals are finding it increasingly difficult to efficiently access and consume pertinent information amidst the vast sea of available content. This issue necessitates the development of effective strategies and tools to streamline the news consumption process, ensuring that users can stay informed without being inundated by excessive, less relevant information.

Additionally, lengthy news articles further compound the issue, they make reading the articles daunting and extremely impractical.

The absence of systematic topic classification exacerbates information overload, hindering users' ability to efficiently locate news articles aligned with their interests.

Supplementarily, news across social media and different websites can lack organization, making it difficult for users to follow topics of interest cohesively. Another thing is the fact that users will often find it challenging to access news from a variety of sources which limits their exposure to different perspectives and makes it exceedingly hard for them to consume news.



Figure 1:Our project Logo

1.2 Goal

The primary goal of this project is to enhance the user experience in accessing and discerning news content within the digital age. By addressing the challenges posed by biased sources, lengthy articles, and the absence of systematic topic classification, the project aims to revolutionize news consumption. The envisioned solution involves the development of a comprehensive system that ensures users have access to concise, accurate, and well-categorized news. Through leveraging advanced technologies, including Natural Language Processing (NLP) and topic modeling, the project seeks to streamline news summarization and categorization processes. Ultimately, the goal is to empower users with an efficient and tailored news consumption experience, enabling them to navigate through information with confidence and discernment

1.3 Objectives

1. Text Summarization: Implement an NLP-based text summarization script to condense news articles into concise summaries, retaining essential information while reducing the overall length, but how can we enhance text summarization?
2. Headline Generation: Use NLP techniques to function as a headline generator that would, given an article, generate an appropriate defining and descriptive headline but how can we use news headline generation as a technique to achieve better summarization?
3. Topic Modeling: Utilize NLP techniques to develop a script for topic modeling, enabling the classification of news articles into relevant categories, thereby enhancing user accessibility to specific areas of interest.
4. Breaking and non-breaking news classification: Design an NLP script to distinguish between breaking and non-breaking news articles, leveraging linguistic patterns and content analysis to provide accurate classification for the articles
5. Integration with Flask: Integrate the web scraper and classification scripts into a Flask application to create a cohesive backend system that facilitates seamless communication between the chatbot and the NLP-powered functionalities.

6. Chatbot Development with Dialog Flow: Develop a conversational interface using Dialog Flow to create an interactive and user-friendly chatbot capable of receiving user queries, retrieving relevant news, and providing summarized, categorized, and authenticated responses. How will using the chatbot as an interface enhance the process of news consumption to users?

1.4 Aim of work

The proposed news analysis system aims to address the challenges of information overload, disorganization, and unreliability in news consumption. To achieve this, a chatbot powered by artificial intelligence and natural language processing techniques will be implemented. This chatbot will assist users in navigating and accessing news content from social media platforms and reputable newspaper websites like CNN and BBC. By leveraging transformer models such as Llama 2, the chatbot will provide personalized and streamlined news consumption experiences, headlines that capture the essence of the original articles and will use BERT summarization to summarize articles. Additionally, the system will utilize BERTopic for topic modeling, enhancing the chatbot's ability to categorize articles based on underlying themes. Furthermore, the system will incorporate transformer-based models like BERT.

The primary objective of this work is to develop a sophisticated chatbot system utilizing Python for backend fulfillment and Dialogflow for natural language processing. The system is designed to enhance user interactions by configuring Dialogflow to define user intents, thereby streamlining the interaction process. The backend fulfillment leverages Ngrok to establish local web servers and maintain seamless connectivity with Dialogflow. A key functionality of this system is its ability to present a curated stream of news articles to users.

The articles are scrapped and processed through our models to ensure they are presented in a user-friendly manner, including links to the original articles, headlines, summaries, topic classifications, and designations as breaking or non-breaking news. The conversational nature of the chatbot promotes user engagement, providing an intuitive interface that aligns with the preferences of contemporary users accustomed to instant messaging platforms. This approach aims to offer a personalized and efficient

news consumption experience, addressing the challenges of information overload and the dissemination of unreliable information.

Overall, this system aspires to deliver a refined, user-centric news delivery mechanism, enhancing the accuracy and relevancy of the information provided to users.

Chapter 2: Background

2.1 Introduction

After deciding on a project, we began researching Artificial Intelligence techniques to improve classification and generation. We identified various algorithms and techniques for Deep Learning.

2.2 Background

After conducting some research, we discovered that in deep learning for text classification, Natural Language Processing (NLP) has high accuracy and faster training. Natural Language Processing (NLP) is a branch of artificial intelligence that focuses on the interaction between computers and humans through natural language. It involves the development of algorithms and models that enable machines to understand, interpret, and generate human language in a way that is both meaningful and useful. NLP applications include language translation, sentiment analysis, chatbots, and speech recognition, among others. NLP involves developing algorithms and models that allow computers to understand, interpret, and generate human language in a meaningful and useful manner.

NLP techniques and algorithms will be used for classification and model training to achieve high accuracy and good predictions, as NLP techniques can help extract relevant information from large volumes of structured/unstructured text data, and NLP models can generate human-like text based on given prompts or summarize lengthy documents into shorter, coherent summaries.

2.3 Deep Learning

Deep learning is a subtype of machine learning in which the neural network has three or more layers. These neural networks seek to replicate the function of the human brain, albeit far from its capabilities, allowing it to "learn" from enormous volumes of data. While a neural network with only one layer can produce approximate predictions,

adding more hidden layers can help to optimize and improve accuracy. It eliminates part of the data pre-processing that is commonly associated with machine learning. These algorithms can ingest and handle unstructured data such as text and photos, and they automate feature extraction, relieving part of the burden on human experts.

Deep neural networks are made up of numerous layers of interconnected nodes, with each layer improving and optimizing the preceding layer's prediction or categorization.

Activation functions introduce non-linearity into deep neural networks, allowing it to learn complex patterns. Common functions include ReLU, sigmoid, and tanh.

Forward propagation refers to the progression of calculations over a network. A deep neural network's input and output layers are referred to as "visible layers." The input layer is where the deep learning model receives data for processing, whereas the output layer is where the final prediction or classification is formed. Dense layers, also known as fully connected layers, are layers in a neural network where each neuron is connected to every neuron in the previous layer

Loss functions measure how well the neural network's predictions match the actual data. Common examples include mean squared error for regression tasks and cross-entropy loss for classification tasks.

Backpropagation is the algorithm used to train neural networks by propagating the error backward through the network. It adjusts the weights of the connections by calculating the gradient of the loss function concerning each weight.

Optimizers are algorithms that adjust the weights of the neural network to minimize the loss function. Popular optimizers like SGD, Adam, and RMSprop.

2.4 Natural Language Processing

Natural language processing (NLP) is a field of computer science, artificial intelligence, and computational linguistics concerned with the interactions between computers and human (natural) languages, recognizing the structure and meaning of words, sentences, texts, and deep understanding of broad language.

Human language is riddled with ambiguities, making it extremely difficult to create software that reliably detects the intended meaning of text or voice input. Homonyms, homophones, sarcasm, idioms, metaphors, grammar and usage exceptions, variations in sentence structure—these are just a few of the irregularities of human language that take humans years to learn, but that programmers must teach natural language-driven applications to recognize and understand correctly from the start if those applications are to be useful.

2.5 Language models

Language modeling is used in artificial intelligence (AI), natural language processing (NLP), natural language understanding, and natural language generation systems, particularly those that generate text, translate words, and answer questions.

The primary goal of Language Modeling is to forecast what word will come next by evaluating text data and calculating word likelihood. They analyze this data by feeding it into an algorithm that generates natural language context rules. The model then uses these rules in language challenges to accurately anticipate or generate new sentences. The program essentially learns the properties and characteristics of basic language and then applies those qualities to comprehend new sentences.

[1]2.6 Transformers

Traditional NLP models, including recurrent neural networks (RNNs), have been widely employed for a variety of NLP tasks. Transformers, on the other hand, have emerged as a powerful alternative, particularly for tasks requiring natural language comprehension and generation.

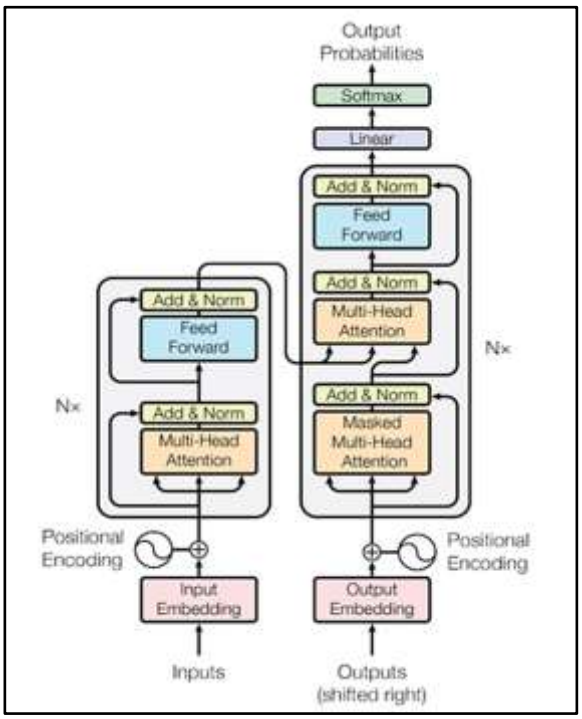


Figure 2: Transformer Architecture

Transformers' central principle is the self-attention mechanism, which allows the model to balance the value of various phrases or tokens in a sequence while making predictions. Transformers, unlike RNNs, do not process inputs in sequence. Instead, they process the full sequence in parallel, which allows for more efficient computing and captures long-term interdependence.

The transformer architecture has two primary components: the encoder and the decoder. The encoder converts an input sequence, such as a sentence, into a series of high-dimensional representations, or embeddings. The encoder's self-attention mechanism enables the model to understand the relationships between the words in the input sequence, The decoder, on the other hand, uses the encoder's output to construct a series of predictions. It also makes use of self-attention, but it also includes another attention process known as "encoder-decoder attention." This attention mechanism assists the decoder in focusing on important areas of the input sequence during the generating phase.

Transformer Architecture
[1] https://arxiv.org/abs/1706.03762

2.7 Large Language Model

Large language models use transformer models and are trained on vast datasets, hence their size. This allows them to detect, translate, predict, or generate text or other material. They are also known as neural networks (NNs), which are computing systems modeled after the human brain. These neural networks use a network of stacked nodes, like neurons.

Large language models are composed of multiple neural network layers. Recurrent layers, feedforward layers, embedding layers, and attention layers work in tandem to process the input text and generate output content.

1. The embedding layer creates embeddings from the input text. This part of the large language model captures the semantic and syntactic meaning of the input, so the model can understand context.

2. The feedforward layer (FFN) of a large language model is made of up multiple fully connected layers that transform the input embeddings. In so doing, these layers enable the model to glean higher-level abstractions — that is, to understand the user's intent with the text input.

3. The recurrent layer interprets the words in the input text in sequence. It captures the relationship between words in a sentence.

4. The attention mechanism enables a language model to focus on single parts of the input text that is relevant to the task at hand. This layer allows the model to generate the most accurate outputs.

2.8 Llama 2

Llama 2 represents a series of pre-trained and fine-tuned large language models (LLMs) developed and released by Meta AI in 2023. These models are available as both foundational models and fine-tuned "chat" models, making them versatile for various Natural Language Processing (NLP) tasks, including text generation.

The Llama 2 model family comprises multiple variants, each differing in parameters such as the number of parameters, token size, learning rate, maximum sequence length, and the maximum number of tokens the model can handle.

Variants of Llama 2 in the Hugging Face Repository

1. **meta-llama/Llama-2-7b-chat-hf**: This variant was utilized for our Headline Generation task. It is specifically tailored for conversational AI applications, such as chatbots and dialog systems. This model variant was chosen due to its training on data that emphasizes conversational patterns and understanding, making it particularly suitable for our news analysis chatbot.
2. **meta-llama/Llama-2-7b-large-hf**: This larger and more powerful variant is ideal for tasks requiring a complex model. However, its enhanced complexity and capability come with increased computational demands. This model is trained on a diverse mixture of publicly available online data, enabling it to handle more intricate language tasks.
3. **meta-llama/Llama-2-7b-medium-hf**: With 7 billion parameters, this variant manages intricate language representations effectively. It offers a balanced compromise between computational resource requirements and performance, making it practical for both research purposes and real-world applications where computational efficiency is a consideration.
4. **meta-llama/Llama-2-7b-small-hf**: This variant is ideal for scenarios with constrained computational resources. It maintains similar features to the larger models while being more manageable in terms of resource consumption.

Given that our chatbot is a conversational dialog system, the meta-llama/Llama-2-7b-chat-hf variant was an ideal choice. This smallest chat model in the Llama 2 family is specifically designed for chat-oriented tasks.

This version of Llama 2 utilizes an improved transformer architecture, which is an evolution of the original transformer model, to effectively process natural language text. Pre-trained on 2 trillion tokens of publicly available text data, it offers a broad understanding of language and context. Furthermore, it has been fine-tuned on over one

million human-annotated instruction datasets, enhancing its ability to comprehend and respond to conversational prompts.

The model contains 7 billion parameters, making it sufficiently large to handle a wide array of language nuances and patterns. Parameters represent the individual weights that learn from the training data and determine how input data is transformed into output, thereby enabling the model to make accurate predictions based on the input received. The number of parameters indicates the model's capacity to capture complex patterns and nuances in the data.

Additionally, the model employs a tokenizer capable of handling a context window size of 4096 tokens, allowing it to maintain context over extended conversations. The tokenizer used is 'transformers.AutoTokenizer.from_pretrained' from the transformers library. Integration with Hugging Face enhances the convenience and ease of developing systems with this model.

2.9 BERT

BERT (Bidirectional Encoder Representations from Transformers) is a revolutionary Natural Language Processing (NLP) model introduced by researchers at Google AI in 2018. BERT has significantly impacted the field of NLP by setting new benchmarks for various NLP tasks and enabling more nuanced and accurate language understanding.

BERT is built upon the transformer architecture, which was introduced by Vaswani et al. in 2017. Unlike traditional models that read text input sequentially (left-to-right or right-to-left), BERT reads the entire sequence of words simultaneously using bidirectional context. This approach allows BERT to capture more comprehensive contextual information from both preceding and succeeding words in a sentence, leading to a deeper understanding of language nuances.

BERT employs a two-stage training process: pre-training and fine-tuning.

1. **Pre-training**: During this stage, BERT is trained on large corpora of text data using two unsupervised tasks:

    - **Masked Language Modeling (MLM)**: In this task, random words in a sentence are masked, and the model is trained to predict these masked words based on the context provided by the surrounding words.
    - **Next Sentence Prediction (NSP)**: This task trains the model to understand the relationship between two sentences. Given a pair of sentences, the model predicts whether the second sentence follows the first one in the original text.
2. **Fine-tuning**: After pre-training, BERT is fine-tuned on specific downstream tasks such as question answering, sentiment analysis, and named entity recognition. This stage involves supervised learning where BERT is trained on labeled datasets relevant to the specific task.

BERT has several variants, tailored for different sizes and purposes:

1. **BERT-Base**: The original model with 12 layers, 768 hidden units, 12 attention heads, and 110 million parameters.
2. **BERT-Large**: A larger version with 24 layers, 1024 hidden units, 16 attention heads, and 340 million parameters.
3. **DistilBERT**: A smaller, faster, and lighter version of BERT that retains 97% of BERT's language understanding capabilities with only 60% of its parameters.
4. **RoBERTa (Robustly optimized BERT approach)**: A variant of BERT that is trained with more data and longer sequences, removing the NSP task, and tweaking key hyperparameters to improve performance.
5. **ALBERT (A Lite BERT)**: A version designed to be lighter and more scalable than the original BERT, using techniques such as parameter sharing across layers and factorized embedding parameterization.

2.10 BERTopic

BERTopic is an advanced topic modeling technique that leverages the power of BERT (Bidirectional Encoder Representations from Transformers) to identify topics in large collections of text data. Introduced by Maarten Grootendorst, BERTopic combines the

contextual embedding capabilities of BERT with traditional topic modeling methods, resulting in more coherent and meaningful topic extraction.

BERTopic utilizes BERT embeddings, which provide rich, contextual representations of words and sentences.

1. **Embeddings with BERT**:
    - BERT is used to generate embeddings for the text data. These embeddings capture the contextual relationships between words, allowing BERTopic to understand the nuanced meanings of terms in different contexts.
    - The embeddings are high-dimensional vectors that represent the semantic content of the text, enabling more accurate topic detection.
2. **Dimensionality Reduction**:
    - The high-dimensional BERT embeddings are reduced in dimensionality using techniques like UMAP (Uniform Manifold Approximation and Projection). This step helps in making the data more manageable while preserving the essential structure and relationships within the data.
3. **Clustering**:
    - Reduced embeddings are clustered using algorithms such as HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise). This unsupervised clustering technique identifies dense regions in the embedding space, which correspond to topics in the text data.
    - HDBSCAN is particularly effective because it does not require the number of clusters to be specified beforehand and can find clusters of varying density.
4. **Topic Representation**:
    - For each cluster, representative terms are identified to describe the topics. This is done by analyzing the terms that contribute most significantly to the embeddings within each cluster.
    - The topics are then labeled with the most representative terms, providing interpretable and coherent topic descriptions.

BERTopic leverages BERT's contextual embeddings for more accurate and meaningful topic extraction compared to traditional methods like LDA, which rely on bag-of-words representations. Unlike traditional methods that require a predefined number of topics, BERTopic's use of HDBSCAN allows for dynamic topic discovery, adapting to the underlying data structure. BERTopic offers flexibility and adaptability, as it can be easily tailored to different languages.

Chapter 3: Methodology and Results

3.1 General Description of the Project

Chatbots are conversational applications that help users interact effectively with systems that facilitate their lives and news is crucial information everyone should know daily. We implemented a system users can use to facilitate their news consumption process giving them a seamless and smooth experience daily. The major goal of the project is to stream analyzed and summarized daily news to users. Through conversation with the chatbot, it will be able to stream news to users, the news will be reliable as they will only be from trusted news sources. The generation portion is that, given a news story, the chatbot will be able to generate a headline and a summary of the given article, making it easier for the user to access all the important information with only a brief glance or skim, it will also be classified as breaking news or not so that the user will find it easier to follow-up only urgent news. Also, the issue genre (topic) will also be demonstrated. By leveraging artificial intelligence and natural language processing techniques, this chatbot aims to provide users with a simplified news consumption experience, mitigating the issues of information overload, disorganization, and unreliability. We used Telegram as the platform in which our chatbot will be integrated, therefore the whole interaction with the system will be through the chatbot making conversations with users where it provides the user with a batch of analyzed news.

To implement the chatbot, we have divided our project into two main parts, the chatbot creation and the model creation/finetuning. In our context, the chatbot creation part is where we used tools such as DialogFlow, flask and ngrok and it is the interface between user questions and reliable responses, while the model's part is where we leveraged LLMs and several other deep learning concepts and methods.

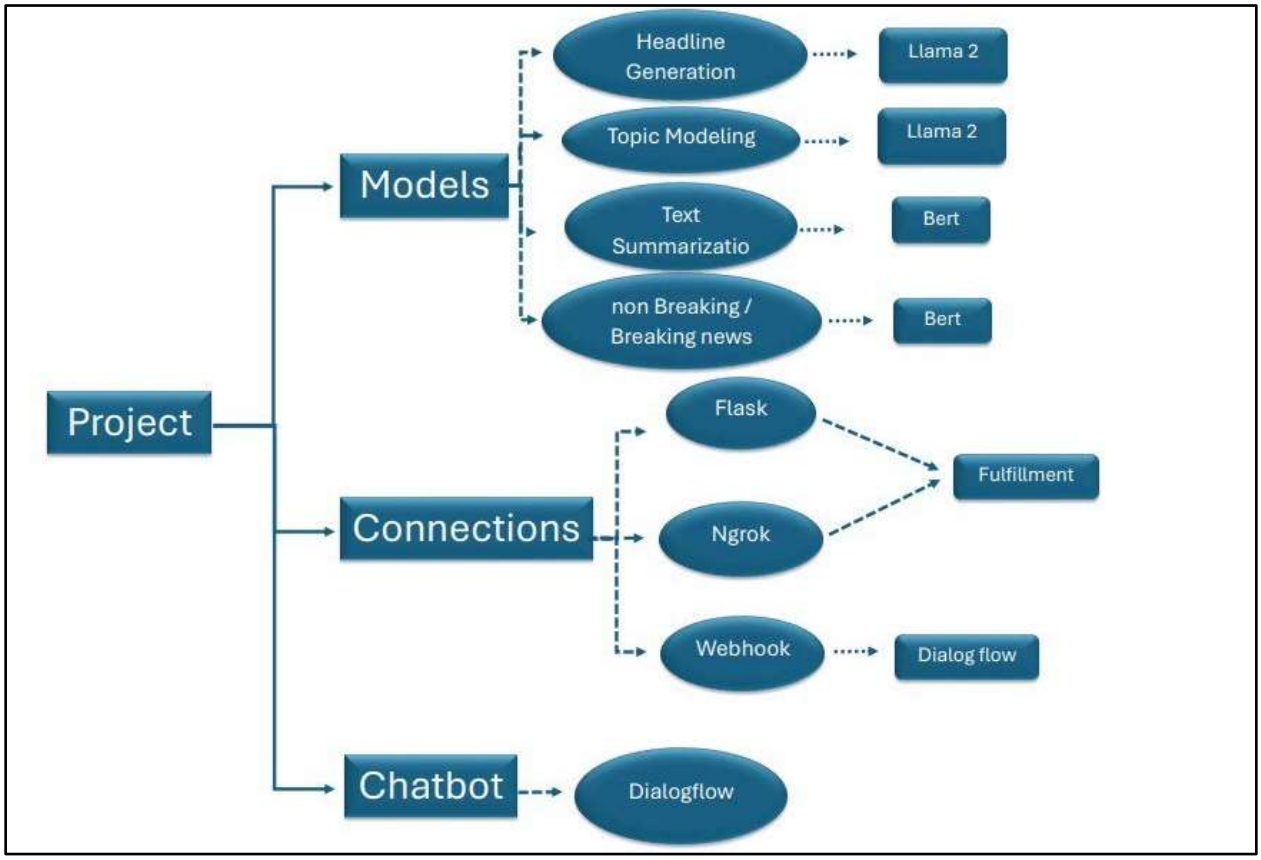


Figure 3: Shows the project components

3.2 Tasks and models

The project aims to streamline news consumption by embedding deep learning and machine learning models to analyze news articles before they reach users. It utilizes topic modeling to identify and cluster similar words, making it easier to categorize articles by topic. In addition, the project includes a breaking news classification system to highlight important, real-time events, and a headline generation feature to create compelling and informative titles for each article. Finally, text summarization is employed to provide concise summaries of key points, helping users quickly grasp the essential information without reading lengthy articles.

3.2.1 Headline Generation

News articles are one of the most popular forms of media as it allows people to get up-to-date accounts of recent events and a significant part of any news article is its headline as it highlights what a news article is mainly about and allows the reader to know beforehand what this article is about. As a result, we chose to employ news headlines to

more efficiently summarize the content for the user, as some articles may have a misleading headline or no headline at all. We implemented a headline generator using Llama2.

3.2.1.2 Dataset

The dataset ‘CNN_Lite_HG’, utilized in our project, is a compilation of news articles sourced from the CNN Lite website. This dataset comprises four columns: Title, URL, Date, and Whole Paragraph. The 'Whole Paragraph' column contains the complete news articles, while the 'Title' column includes the corresponding headlines. Although the 'URL' and 'Date' columns were not directly utilized in our analysis, they provide the URLs and publication dates for each news article, respectively.

To build and update this dataset, we initially searched for websites offering both articles and headlines. After extensive research, we identified CNN Lite as a reliable source with excellent headlines. Using the BeautifulSoup Python library, which is particularly effective for web scraping, we extracted the headlines and links from the website. We then iterated through these links, scraping each article and associating it with its corresponding headline. Below in Figure kaza is a snippet of the dataset

Title	URL	Date	Whole Paraghraph
HIV/AIDS a	https://lite	########	Longtime LGBTQ+ activist David Mixner, who was a leader in the fight for social change a
RFK Jr. has	https://lite	########	Independent presidential candidate Robert F. Kennedy Jr. has selected a vice presidenti
Trump tear	https://lite	########	Donald Trumpâ€™s defense team says it will emphasize his past lawyersâ€™ involvemen

Table 1:shows data used in Headline generation

We also chose to use Google Colaboratory to develop our headline generator because it provides enough VRAM and GPU for our task.

3.2.1.3 Model selection

Llama 2 is a series of large language models (LLMs) developed by Meta AI in 2023, designed for various NLP tasks including text generation. These models, available on Hugging Face, come in multiple variants differing in parameters such as size, learning rate, and sequence length.

Llama 2 'meta-llama/Llama-2-7b-chat-hf' employs an advanced transformer architecture, pre-trained on 2 trillion tokens, and fine-tuned on over one million human-annotated datasets. With 7 billion parameters and a tokenizer capable of handling 4096 tokens, it effectively processes and maintains context in extended conversations. Integration with Hugging Face simplifies development.

Quantization

Although the chat version of the Llama2 model is small compared to other variants in the family, it is still large enough to not run efficiently on our computers since they are not equipped with powerful GPUs and cannot handle large computations that come with the Llama2 models, so we used **quantization** to load the large model with less GPU.

To load the 7 billion parameter model, we need to perform some methods of optimization, especially because we have limited VRAM, so we need to condense the model a little bit so that it runs smoothly acquiring less space.

More specifically, we have used the 4-bit quantization from the bitsandbytes library. 4-bit quantization is exceptionally useful as it reduces the 64-bit representation to only 4-bits which reduces the GPU memory. This is used instead of the original 32-bit which gives us speed and reduces memory.

Model Loading

To use the model, we first loaded it using hugging face and the transformers library then we loaded the Llama2 tokenizer. Tokenizers are a crucial component in the preprocessing pipeline of our task as they convert raw text into numerical tokens that the Llama2 model can understand and process and then converts the model's output

tokens back into human-readable text. The Llama2 model tokenizer is a BPE model based on sentencepiece.

Llama 2 Prompt Engineering

After loading both the model and the tokenizer, we were able to leverage the Llama2 power by prompting it. Because Lllama2 Chat is an open conversational model, we needed to interact with it by providing appropriate prompts and questions in order to generate logical and coherent responses for effective use.

We can directly prompt the system with our request but meta has created a template for promoting the model which we followed. The prompt template is shown in the figure below.

```
"""
<s>[INST] <<SYS>>

{{ System Prompt }}

<</SYS>>

{{ User Prompt }}
 [/INST]

{{ Model Answer }}
"""
```

Figure 4:shows lama 2 system prompts

There are two main components in the template, namely the **system prompt** and the **user prompt.**

- System prompt is for guiding the model during the conversation.
- User prompt is how we ask a question.

The system prompt is the information given to all conversations and for our model we have prompted it saying, “You are a very smart unique news article headline generator”.

The user prompt has a lot more details and will involve more information, as it will consist of two main components: an example **prompt** and a **main prompt**. Since most large language models perform a lot better with generating accurate responses when given an example to work with, we have created the example prompt with an example of an article, and we have given it the headline of the article as well to show it an accurate example of the kind of output we are expecting from it. Next is the main prompt where we add the main tags.

The generator function was utilized in a loop to generate and display headlines for a series of articles. Inside it, the constructed prompt is passed to a text generator model to produce a headline. The generated text is parsed to extract the headline, ensuring any extraneous tags are removed. If the expected delimiter for headlines is not found, the function gracefully handles the error by returning a default message indicating that no headline was generated. This function was utilized in a loop to generate and display headlines for a series of articles in the dataset 'CNN_Lite_HG'.

Results

To test the headline generator, we employed a qualitative evaluation method. We conducted a manual review of the generated headlines and compared them to the original headlines from the 'CNN_Lite_HG' dataset. We have manually reviewed each generated and true headline.

After executing the generator function on the entire dataset, we compiled all the generated headlines into one column of an empty CSV file. The corresponding original headlines were placed in another column, labeled 'candidate headlines' and 'reference headlines' respectively. Additionally, a third column named 'Scores' was added to record the score for each individual row.



Candidate Headlines	Reference Headlines	Scores
'Trailblazing LGBTQ+ Activist David Mixner Dies, Leaves Legacy of Social Change and Equality'	HIV/AIDS activist David Mixner dies at 77 1	1
'RFK Jr. Set to Announce Vice Presidential Nominee Within Weeks, Eyes Former Governors, Quarterbacks and Civil Rights Attorneys for Spot'	RFK Jr. has chosen VP nominee who he will announce within two weeks	1
'Trump's Legal Team Set to Challenge Hush Money Charges, Emphasizing Past Lawyers' Involvement'	Trump team says it's argue lawyersâ€™ involvement in alleged hush	1
'Age vs. Policy: Can Biden's Vulnerabilities Help Republicans Win the White House?'	Opinion: Her hearing shows GOP will need more than age issue against I	1
'TikTok Creators Unite Against Bill Threatening App's Future in US, Highlighting Economic Impact and Community Building'	TikTok creators fear a ban as the House prepares to vote on a bill that co	1

Table 2: Shows data used for testing headline generation

The 'Scores' column contained the evaluation for each headline, with a score of '0' if the candidate (generated) headline did not match the reference (original) headline, and a score of '1' if it did. We individually assessed both headlines and compared them. If the headlines conveyed the same meaning, were free from spelling mistakes, hallucinations, and grammatical errors, the row was assigned a score of '1'; otherwise, it received a score of '0'.

Only 674 headlines were tested. The results were:

- Out of the 674 headlines, 516 had scored '1'
- Out of the 674 headlines, 158 had scored '0'

Using the accuracy formula:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{total}}$$

Equation 1 Accuracy Formula

We have concluded that the model had 0.765% accuracy.

3.2.2 Breaking News Classification

The rushed flow of news can overwhelm our minds, resulting in information fatigue and decreased focus. Excessive consumption frequently wastes important time that may be used more efficiently. Furthermore, it can increase anxiety and make it difficult to distinguish between crucial and inconsequential updates.

In this section we discuss the implementation of a Breaking news classification model that works to assist the user to read only crucial news without wasting time on other trivial ones. To save time and stay up to date with significant global news, this model only provides the most important stories. To build the classifier, we used multiple technologies and methods.

3.2.2.1 Dataset

The absence of commercially available datasets specifically tailored for classifying breaking news, as observed across various data sources such as Kaggle, UCL Machine Learning Repository, Google Dataset Search, and Hugging Face, necessitated the creation of a custom dataset. To address this need, we employed web scraping techniques to collect breaking news headlines, utilizing Python libraries and Twitter as a primary source for the data collection process.

Former Barcelona footballer Dani Alves found guilty	Breaking_News
Public is at risk because recommendations to impr	Breaking_News
Speaker Sir Lindsay Hoyle says sorry to UK MPs afte	Breaking_News
Heather McDonald's 'Juicy Scoop' Podcast Seals A	Non_Breaking_News
How 'The Conners' Went From Nearly DOA to a Mile	Non_Breaking_News

Table 2:shows data used in Breaking and non_breaking classification

Breaking news refers to events that are currently unfolding or "breaking," typically involving unforeseen incidents such as fires or plane accidents. These events are of significant relevance and interest to the general public and are covered instantly by news outlets. Consequently, the news items included in the breaking news dataset we created consist of events that are still developing, have just occurred, and are of broad interest. This criterion formed the basis for distinguishing between breaking and non-breaking news stories.

X (formerly Twitter) is widely recognized as a popular and free news source that millions of people use daily for news consumption. Due to its popularity, X hosts numerous prominent newspapers and news channels, including CNN and BBC, each with a dedicated account for breaking news. We leveraged these sources by scraping their posts using the "ntscraper" library, which includes "Nitter." Given Nitter's limitation to 850 posts per news account, we systematically scraped 850 posts over various periods from diverse breaking news accounts.

For gathering non-breaking news, we employed a different approach, initially identifying websites that publish complementary or entertainment articles. We then compiled a list of links from these websites to facilitate scraping. We utilized Python's BeautifulSoup

library, which is designed for scraping HTML and XML files, along with the HTTP request library, requests. By using the website's inspect function, we identified the specific web page elements required for extraction from the CSS, ensuring the web scraper targeted the correct elements.

Manual data annotation was done to ensure the accuracy of the selected news, as we did this after each scraping time to make sure that the articles we scraped with different topics and to make the data more balanced to avoid biases of training data

Occasionally, manual scraping was necessary. The primary reason was the need to acquire news promptly, requiring us to read each article, classify it, and add it to an Excel sheet. Additionally, most breaking news items were political, with none present in the non-breaking news portion of the data. To avoid bias, we manually selected some trivial, non-critical political news items.

The collected data was initially very unclean, containing tweets and insignificant articles, necessitating a thorough cleaning process. First, we addressed null values and duplicates, eliminating them to ensure data integrity. We then performed a value count to ascertain the number of data points in each class and to check for balance.

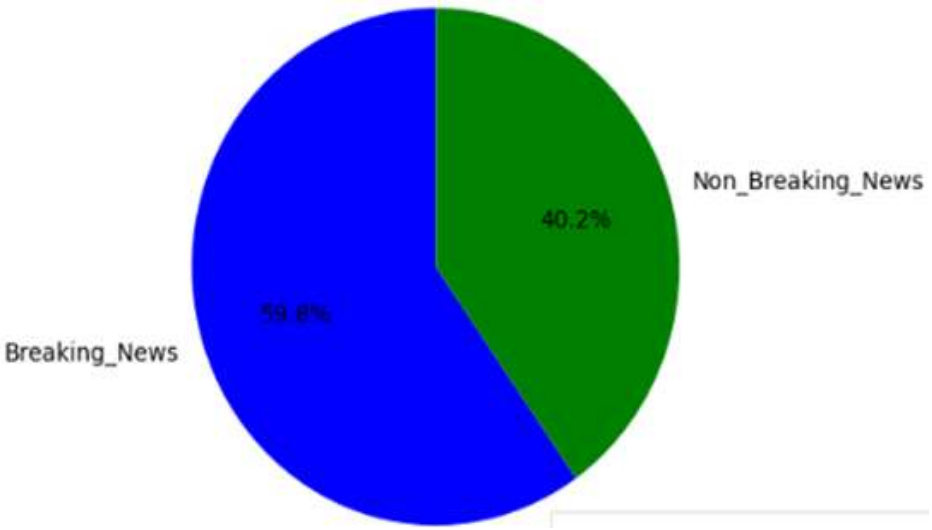


Figure 5:shows Lables percentage in Breaking and non_breaking classification data

Subsequently, we developed a comprehensive cleaning function. This function included methods to remove identities and mentions from tweets, ensuring the data was standardized. Additionally, we created another function to expand and eliminate slang

and informal abbreviations. Before handling contractions, the function verified their existence in the dictionary to ensure accurate expansion.

We have performed text cleaning by grouping together a collection of regex patterns that looks for:

- Punctuation
- HTML tags and attributes
- URL and whitespaces
- Emojis
- usernames and mentions
- converts to lower text

and replaced them with spaces to ensure their removal. Account mentions were also specifically removed. Furthermore, we eliminated stop words to make the data as comprehensible as possible for the model.

Truecasing was implemented to address the limitations of converting all text to lowercase, which can result in the improper handling of proper nouns that should be capitalized. Truecasing capitalizes words that are typically capitalized, ensuring that the text maintains the correct capitalization for proper nouns and other relevant words.

For exploring the dataset, we used word clouds. Using word clouds in EDA of text data is a good idea because it provides a visually intuitive way to understand the most frequent words or phrases in the dataset. Word clouds can quickly highlight prominent themes, topics, or keywords. Moreover, word clouds are easy to interpret and communicate, making them useful for both data exploration and presentation purposes. So we have made a word cloud.

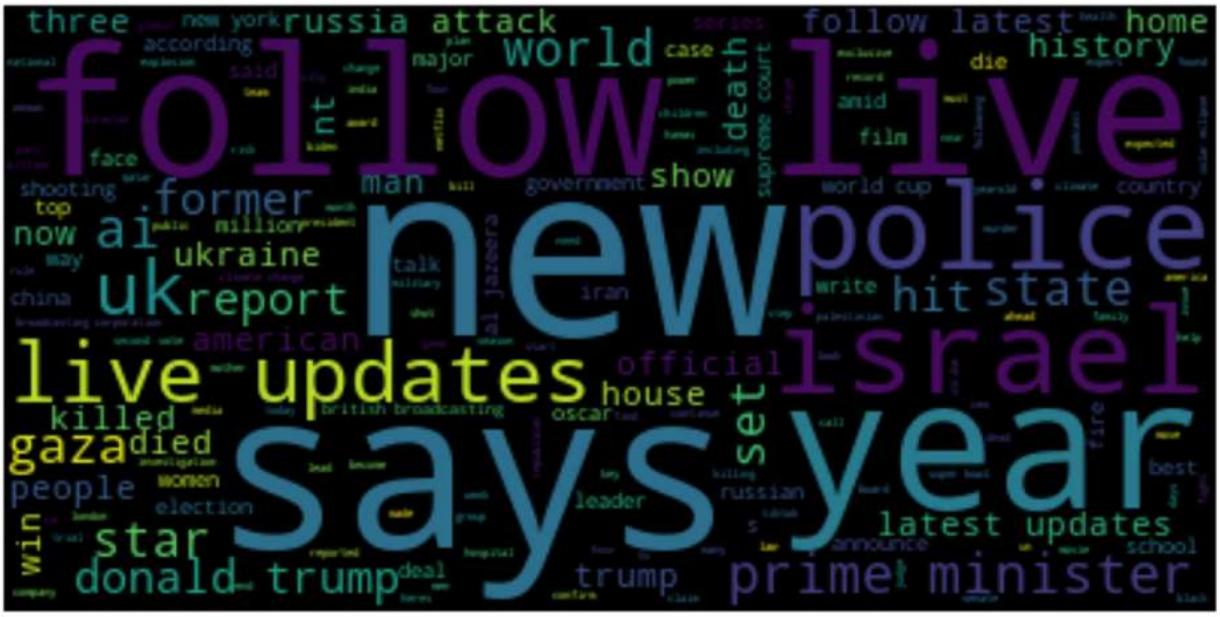


Figure 6: Shows a word cloud

3.2.2.2 Model Selection

Our text classification task involves categorizing news articles into breaking news or non-breaking news categories. BERT (Bidirectional Encoder Representations from Transformers), developed by Google, stands out as a leading natural language processing model renowned for its ability to comprehend word context by considering surrounding words in both directions. This makes BERT highly suitable for our classification task. Due to limited computing resources, we opted for a simplified variant of BERT known as bert-en-uncased-l-6-h-512-a-8.

The model we used is the uncased version of BERT meaning that the model uses uncased text, it does not distinguish between uppercase and lowercase letters. This can simplify the model and reduce its complexity, making it more efficient for usage on less computational power.

The original BERT model has 12 BERT-base layers or even 24 layers but the version we chose has only 6 layers (transformer blocks) therefore it's smaller and faster.

The dimensionality of the encoder layers and the pooler layer is 512, which is also the size of the vectors that represent each token in the input text.

The number of attention heads in the multi-head attention mechanism is 8. Each attention head allows the model to focus on different parts of the sentence and having 8 heads provides a balance between model complexity and the ability to capture intricate patterns in the data.

All in all, this version is compact and efficient enough to provide strong performance on NLP tasks while being less resource-intensive than the full-sized BERT models.

The classifier model is constructed using TensorFlow and leverages pre-trained BERT embeddings to create a reliable text classification system. The architecture is defined as follows:

The model begins with an input layer that accepts raw text input. This is represented as a tf.keras.layers.Input layer with the data type specified as string (dtype=tf.string).

The input text is then passed through a preprocessing layer implemented using a TensorFlow Hub KerasLayer (hub.KerasLayer). This layer applies necessary text preprocessing steps such as tokenization, lowercasing, and adding special tokens required by the BERT model.

The preprocessed text is fed into a BERT encoder layer, also implemented as a TensorFlow Hub KerasLayer. This layer utilizes a pre-trained BERT model to generate contextualized embeddings. The encoder is set to be trainable (trainable=True), allowing fine-tuning during the training process. The BERT encoder outputs two types of embeddings: sequence output and pooled output. For this model, the pooled output (a fixed-size representation of the input sequence) is used.

The pooled output from the BERT encoder is passed through a series of fully connected (dense) layers with ReLU activation functions. Dropout layers are added between dense layers to prevent overfitting. Specifically:

- The first dropout layer has a dropout rate of 0.5.
- The first dense layer reduces the dimensionality to 128 units with ReLU activation.

- A second dropout layer with a dropout rate of 0.3 follows.
- A second dense layer further reduces the dimensionality to 64 units with ReLU activation.

Finally, the output layer is a dense layer with a single unit and a sigmoid activation function. This configuration is suitable for binary classification tasks, as the sigmoid function outputs a probability value between 0 and 1, representing the likelihood of the positive class.

The model compilation step involves specifying the optimizer, loss function, and evaluation metrics, which are essential for training the model effectively.

The Adam (Adaptive Moment Estimation) optimizer is chosen for this model. Adam is an extension of stochastic gradient descent that computes adaptive learning rates for each parameter. It combines the advantages of two other popular optimization methods: AdaGrad and RMSProp.

The learning rate is set at 0.00001 (1e-5). This small learning rate is suitable for fine-tuning pre-trained models like BERT. A lower learning rate ensures that the model weights are adjusted gradually, preventing drastic changes that could disrupt the learned representations from the pre-trained model.

The loss function used is binary cross-entropy, which is appropriate for binary classification tasks. Binary cross-entropy measures the difference between the predicted probability and the actual class label (0 or 1).

The accuracy metric is used to evaluate the model's performance during training and testing. Accuracy is the ratio of correctly predicted instances to the total instances. It provides a straightforward measure of how often the model's predictions are correct

3.2.2.3 Results

After several tests, we finally got these results. The model is not perfect, but the results are excellent as in the training

- Training {accuracy: 0.9917, loss: 0.0318}
- Validation {accuracy: 0.9356, loss: 0.2779}
- test {accuracy: 0.9442, loss: 0.2025}

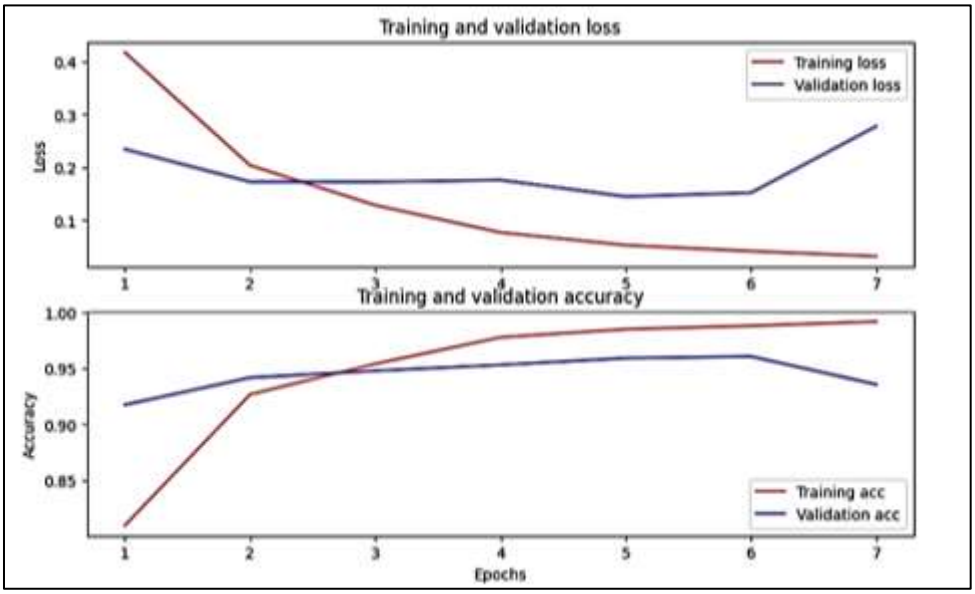


Figure 7: shows the graph of accuracy and lose of Breaking and non-breaking classification model

Confusion matrix

The model shows strong performance, correctly classifying the majority of instances, with relatively low counts of misclassifications.

True Positives (TP): The model correctly predicted 908 instances as "Breaking News."

True Negatives (TN): The model correctly predicted 649 instances as "Non-Breaking News."

False Positives (FP): The model incorrectly predicted 44 instances as "Breaking News" when they were actually "Non-Breaking News."

False Negatives (FN): The model incorrectly predicted 48 instances as "Non-Breaking News" when they were actually "Breaking News."

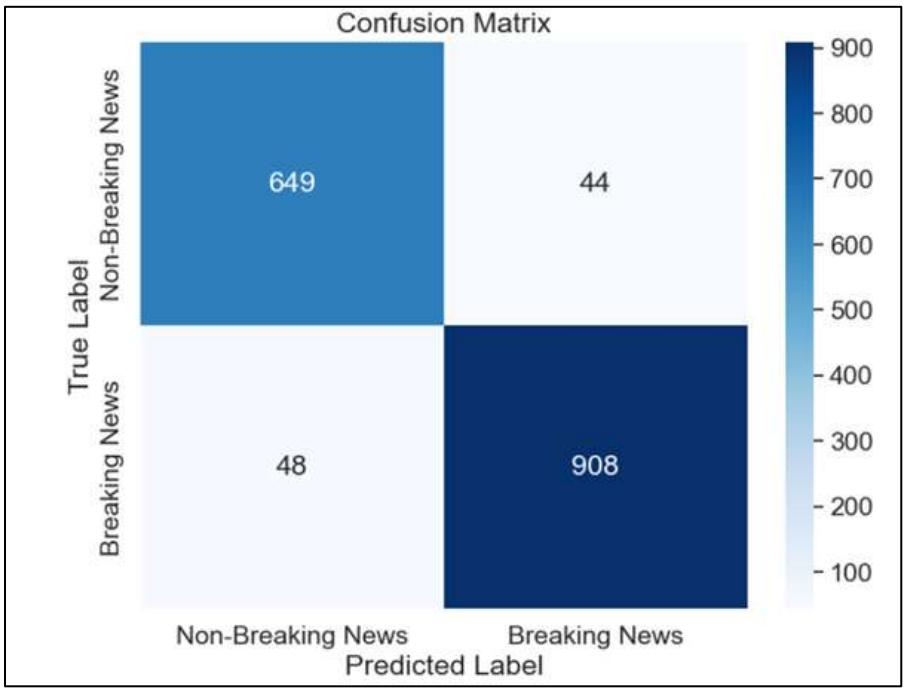


Figure 8:shows confusion matrix of Breaking and non-breaking classification model

3.2.3 Topic Modeling

The vast array of news sources and their daily uploads contribute significantly to information overload, making it challenging for users to find news that aligns with their interests. Social media, being the primary news source, presents hundreds of thousands of news items that lack systematic classification, exposing users to a wide variety of news, including those outside their interests, leading to inefficient time use.

To address this problem, we employ topic modeling, a machine learning technique that uncovers hidden thematic structures within a collection of documents, identifying and organizing themes or topics within large text collections. One of the most effective topic modeling techniques is BERTopic, a deep learning approach that leverages BERT embeddings to create dense, high-dimensional text representations, which are then clustered to identify topics.

3.2.3.1 Dataset

The ‘abc-news-data’ [2]dataset, frequently utilized in various topic modeling tutorials, comprises two columns: 'publish_date' and 'headline_text'. Containing 1,103,663 data points, we applied a train_test_split to randomly divide the dataset, ensuring a diverse topic selection with a 3:2 ratio. From this, we selected a smaller subset for our training data, resulting in a training data size of 44,147.

We employed a text cleaning function that utilizes a collection of regex patterns to identify and remove unwanted elements, such as:

- Punctuation
- HTML tags and attributes
- URLs and whitespace
- Emojis
- Usernames and mentions

Abc-news-dataset
[2] https://www.kaggle.com/datasets/keitazoumana/abcnewsdata

- Converts text to lowercase

This function, named clean_tweet(), replaces these elements with a blank space, effectively cleansing the text. Additionally, we implemented a specific function to remove usernames and mentions from tweets. Both functions were applied to the tweet column, and the cleaned text was stored in a new column named headline_text.

3.2.3.2 Model Selection

We selected BERTopic as our topic modeling technique. Utilizing Llama2, we developed a model that generates topics and classifies each article into one of these topics. BERTopic combines transformer-based models with TF-IDF (term frequency-inverse document frequency) to produce dense clusters that are easy to interpret while preserving significant words in the topic description. This deep learning approach supports the sentence-transformers model.

The BERTopic modeling technique follows five sequential main steps: document embeddings, reducing embeddings in dimensionality, embeddings clustering, and tokenizing documents per cluster and finally extracting the best-representation words per topic.

Using a set of independent words per topic as a cluster and labeling them is not highly efficient; hence, we incorporated the Llama2 model. We utilized the clusters and topics generated by BERTopic and had Llama2 fine-tune and distill that information into more precise representations. Thus, we employed both BERTopic for topic creation and Llama2 for topic representation. The figure below shows an overview of all the algorithms used and why they're used.

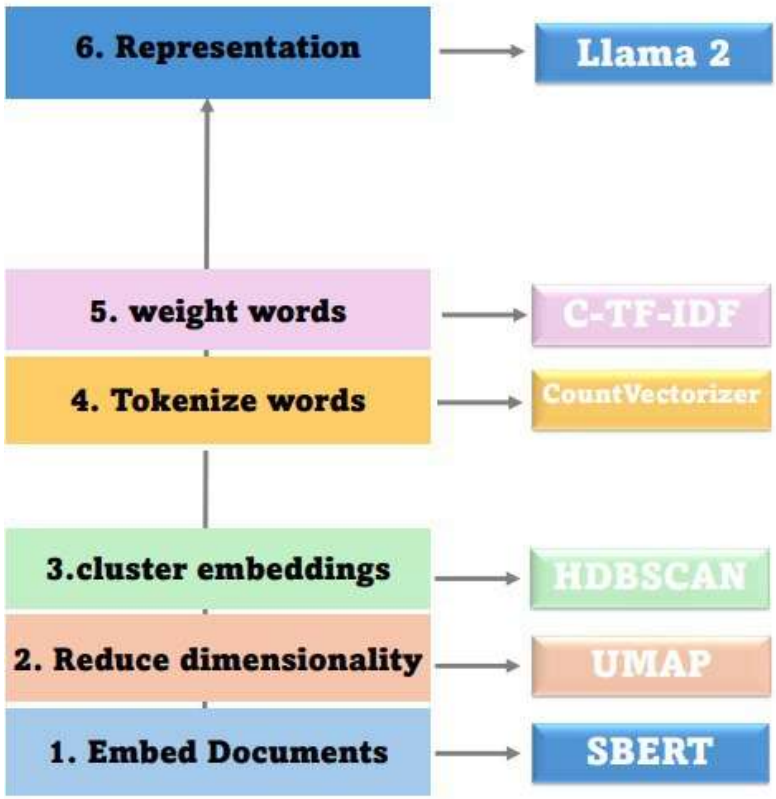


Figure 9 shows the algorithms and sequence used for topic modeling

Document Embeddings and dimensionality reduction using SBERT and UMAP

A deep learning model, or any model, primarily recognizes numbers; thus, documents must first be converted into vectors (embeddings). BERT is one of the most effective methods for generating semantic and context-aware vectors. The default embedding used by BERTopic is the Sentence Transformer, which produces fixed-length vectors for each input, whether a sentence or a paragraph. BERTopic employs SBERT (Sentence BERT) to compute the embeddings for sentences and represent them in vector space, enabling semantic comparison and clustering by similarity.

In high-dimensional space, the concept of spatial locality becomes less defined, and distance measures vary minimally. Dimensionality reduction in a vector space of document embeddings involves simplifying high-dimensional vectors while preserving their semantic relationships. Reducing the dimensionality of embeddings enhances clustering performance. We implemented UMAP to preserve the local and global

features of high-dimensional data in lower projected dimensions, thereby reducing the dimensionality of document embeddings generated from SBERT.

Document Clustering using HDBSCAN

Subsequently, the reduced embeddings were clustered using HDBSCAN. HDBSCAN is a clustering algorithm that identifies clusters with varying densities and effectively manages noise by creating and condensing a hierarchy of clusters based on their stability. This algorithm ensures that unrelated documents are not assigned to any cluster.

The UMAP algorithm is used to reduce the high-dimensional data to a lower-dimensional space. Specifically, the UMAP object is instantiated with parameters n_neighbors=15, n_components=5, min_dist=0.0, metric='cosine', and random_state=42, which configures it to create a 5-dimensional representation of the data while preserving the local and global structure. Additionally, a separate UMAP instance with n_components=2 is used to further reduce the embeddings to two dimensions for visualization purposes.

For clustering, the HDBSCAN algorithm is utilized. The HDBSCAN object is configured with parameters min_cluster_size=150, metric='euclidean', cluster_selection_method='eom', and prediction_data=True. This setup allows for the identification of clusters of varying densities within the data, ensuring that even small clusters are detected while using the Euclidean distance metric to measure similarity.

The pre-reduction step is conducted by applying UMAP to transform the embeddings into a two-dimensional space.

Clusters Representation

Representing the topics is done using several algorithms to get broader insights of the documents. Our primary representation model is c-TF-IDF (Class-based Term Frequency-Inverse Document Frequency), which provides a robust foundation for understanding topic structures. Additionally, we incorporate three other representation

models—KeyBERT, Maximal Marginal Relevance (MMR), and Llama 2—to enrich our topic representations by providing alternative angles and insights.

As previously mentioned, the 'meta-llama/Llama-2-7b-chat-hf' model is sufficiently large to yield interesting and useful results yet compact enough to operate within our environment but still not small enough for our constrained-hardware. Therefore, due to our limited VRAM and GPU resources, we needed to condense the model to ensure it could run efficiently. This was achieved through quantization.

Quantization

Quantization reduces the 64-bit representation to 4-bit, significantly decreasing the GPU memory required. This technique, often employed in machine learning, minimizes the size and computational demands of neural network models. By converting the weights and activations of a model from higher precision (such as 32-bit or 16-bit) to 4-bit representations, the model's memory footprint is markedly reduced, resulting in faster inference and lower power consumption.

Next, we loaded the model and tokenizer then generated a HuggingFace pipeline to allow us to easily generate new text.

Prompt Engineering

To efficiently use the Llama2 model, we need to properly prompt it for the task of giving label representations for the topics (clusters). We can directly prompt the model, but we followed the prompt template provided by meta. The template consists of a system prompt and a user prompt. The system prompt guides the model during a conversation and the user prompt is where we ask it a question. The user prompt has much more details and is divided into two prompts, the main and the example prompts (see examples in appendix A).

c-TF-IDF in BERTopic is an adjusted version of the traditional TF-IDF algorithm, designed to work on a cluster or topic level rather than on a document level. This

method is crucial for BERTopic's ability to generate coherent and interpretable topics from a collection of documents.

KeyBERT is a minimalist keyword extraction technique using BERT embeddings to identify relevant words and phrases by generating document embeddings, creating n-gram candidate keywords, and ranking them based on cosine similarity. It is employed in our model as an additional representation.

Maximal Marginal Relevance (MMR) balances relevance and diversity in information retrieval by selecting items that are highly relevant and minimally similar to already chosen items, reducing redundancy. It is used in our project to ensure relevance and diversity in document selection.

Llama2 is used to represent the topics as comprehensive labels so that it's easier to represent the clusters as words.

In our project we employed the algorithms as follows: KeyBERT focuses on identifying keywords within texts, enhancing the interpretability of topics through key terms. Maximal Marginal Relevance (MMR) balances relevance and diversity, ensuring that the selected terms are both highly relevant to the topic and diverse enough to cover different aspects of it. Llama 2 is employed to generate descriptive labels for the topics, and provide coherent and meaningful topic summaries.

We first initiated a 'BERTopic' object with the 4 sub-models we explained previously and hyperparameters. The key hyperparameters are 'top_n_word' which specifies the number of top words to describe each topic and 'verbose=true', which enables detailed logging of a model's operations.

The fit_transform method is used to fit the model to the data and transform the documents from our dataset into topics and their corresponding probabilities.

4.2.3.3 Results

After training the BERTopic model on a collection of news headlines, the get_topic_info() method is used to retrieve and display a comprehensive summary of the identified topics. This method outputs a DataFrame containing various pieces of information for each topic.

	Topic	Count	Name	Representation	KeyBERT	Llama2	MMR	Representative_Docs
0	-1	10234	-1_new_says_govt_council	[new, says, govt, council, country, plan, hour...	[new, australia, qld, government, govt, countr...	[Government and politics, ..., ...]	[new, says, govt, council, country, plan, hour...	[council launches group plan youth training, t...
1	0	7438	0_police_man_crash_court	[police, man, crash, court, charged, murder, st...	[killed, stabbing, accident, murder, injured, ...	[Crime, ..., ...]	[police, man, crash, court, charged, murder, d...	[sydney man killed north west car crash, police...
2	1	4985	1_cup_win_world_final	[cup, win, world, final, wins, afl, england, t...	[socceroos, aussies, wins, rugby, final, nrl, ...	[Sports, ..., ...]	[cup, win, world, final, wins, afl, england, t...	[australia win first match new coach lehmann, ...
3	2	1820	2_iraq_killed_us_troops	[iraq, killed, us, troops, afghan, un, iraqi, ...	[iraq, iraqi, baghdad, soldiers, fallujah, tro...	[War and Conflict, ..., ...]	[iraq, killed, us, troops, afghan, un, iraqi, ...	[us troops spend longer iraq, us soldier kille...
4	3	1703	3_farmers_drought_cattle_growers	[farmers, drought, cattle, growers, dairy, bee...	[farmers, farming, farmer, agriculture, agricu...	[Agriculture and Farming, ..., ...]	[farmers, drought, cattle, growers, dairy, bee...	[farmers reconsidered drought aid, new farmers ...
5	4	1508	4_election_labor_rudd_trump	[election, labor, rudd, trump, gillard, libera...	[electoral, electorate, election, elections, v...	[Politics, ..., ...]	[election, labor, rudd, trump, gillard, libera...	[us election electoral college vote donald tru...
6	5	1139	5_market_profit_share_rates	[market, profit, share, rates, wall, dollar, b...	[shares, sharemarket, trading, investors, mark...	[Finance, ..., ...]	[market, profit, share, rates, wall, dollar, b...	[australia share market open, australian share...
7	6	1111	6_festival_art_film_show	[festival, art, film, show, one, music, day, n...	[festival, celebrations, celebrates, melbourne...	[Arts and Culture, ..., ...]	[festival, art, film, show, one, music, day, n...	[vanuatu art show canberra, melbourne music fe...
8	7	1034	7_health_hospital_cancer_doctors	[health, hospital, cancer, doctors, patients, ...	[hospitals, patients, hospital, care, health, ...	[Healthcare, ..., ...]	[health, hospital, cancer, doctors, patients, ...	[mental health service faces funding fears, pa...
9	8	990	8_fire_blaze_bushfire_fires	[fire, blaze, bushfire, fires, firefighters, h...	[fires, fire, firefighters, bushfire, bushfire...	[Fire and emergency services, ..., ...]	[fire, blaze, bushfire, fires, firefighters, h...	[fire crews continue battle blaze qld island, ...
10	9	756	9_fishing_fish_dog_feral	[fishing, fish, dog, feral, dogs, reef, rspca, ...	[fishing, fisheries, anglers, fishers, fish, w...	[Fishing regulations, ..., ...]	[fishing, fish, dog, feral, dogs, reef, rspca, ...	[anglers prepare new fishing season, govt plan...
11	10	679	10_interview_speaks_media_extended	[interview, speaks, media, extended, interview...	[interview, interviews, discusses, talks, spea...	[Interviews, ..., ...]	[interview, speaks, media, extended, interview...	[extended interview bill south, interview jame...
12	11	615	11_flood_storm_weather_cyclone	[flood, storm, weather, cyclone, floods, rain, ...	[floods, flooding, floodwaters, flooded, flood...	[Flood Relief, ..., ...]	[flood, storm, weather, cyclone, floods, rain, ...	[flood fears far north coast, north qld get fl...
13	12	549	12_council_mayor_councillor_councils	[council, mayor, councillor, councils, housing,...	[mayor, council, councils, councillors, mayora...	[Local Government, ..., ...]	[council, mayor, councillor, councils, housing,...	[break day mayor defends council bushfire, ex ...
14	13	520	13_mine_mining_coal_miners	[mine, mining, coal, miners, miner, gold, ore,...	[mining, miners, mines, coal, bhp, miner, ore,...	[Mining, ..., ...]	[mine, mining, coal, miners, miner, gold, ore,...	[coal mine hold amid protests, call meeting pr...
15	14	511	14_school_teachers_education_schools	[school, teachers, education, schools, student...	[school, schools, schoolies, education, classr...	[Education, ..., ...]	[school, teachers, education, schools, student...	[pay attention education teachers union, educa...
16	15	444	15_rail_highway_transport_train	[rail, highway, transport, train, ...	[rail, railway, cityrail, trains, ...	[Transportation, ..., ...]	[rail, highway, transport, ...	[elic rail upgrade funding, murray ...

Table 3: shows topic modeling data used, bertkey and clusters

The DataFrame produced by get_topic_info() includes several columns, each providing specific details about the topics:

- **Topic**: A unique identifier for each topic.
- **Count**: The number of documents assigned to each topic
- **Name**: An automatically generated name for each topic, summarizing its key terms.
- **Representation**: The top words representing each topic
- **KeyBERT**: Keywords identified using the KeyBERT model

- **Llama2**: Topic labels generated using the Llama 2 text generation model, providing coherent and descriptive names for the topics.
- **MMR**: Terms selected using Maximal Marginal Relevance (MMR)
- **Representative_Docs**: Example documents that best represent each topic

This table provides an organized and detailed overview of the topics. It highlights the number of documents per topic, key terms, and representative documents, making it easier to understand and interpret the model's outputs.

For example, the table provides detailed information about Topic 7, which comprises 1,034 documents. The topic is labeled "7_health_hospital_cancer_doctors", signifying its central themes. The Representation column includes keywords such as "health," "hospital," "cancer," "doctors," and "patients," indicating a strong focus on healthcare and medical issues. The KeyBERT column supplements this with additional relevant terms like "hospitals," "patients," "hospital," "care," and "health," reinforcing the medical and healthcare context. The Llama2 column categorizes this topic under "Healthcare", succinctly encapsulating its primary focus.

Lastly, the Representative_Doc's column provides examples of documents associated with this topic, including phrases such as "mental health service faces funding fears," which offer concrete illustrations of the topic's relevance within the dataset. The top 10 words are shown in the figure below:

```
[('hospitals', 0.9136259),
 ('patients', 0.9077384),
 ('hospital', 0.9031883),
 ('care', 0.88824356),
 ('health', 0.8871986),
 ('emergency', 0.88614124),
 ('patient', 0.88478),
 ('nurses', 0.88396657),
 ('nursing', 0.8773122),
 ('urged', 0.8769383)]
```

Figure 10 Shows the 10 top words of topic 7

3.2.3 Text summarization

Informative and well-structured news articles are most likely extremely lengthy and sometimes even full of advanced vocabulary; consequently, users wanting to read news daily will spend so much time reading filler paragraphs that do not necessarily include the most important information which in return wastes the user so much time. Text summarization is a Natural Language Processing task that extracts the most important information from a long document and generates a simpler shorter version of the document. To our advantage, the summarized text is acutely shorter and retains only the significant notions and information in a long text. We needed a model capable of accurately extracting the key points while avoiding repetitive information. Luckily, contemporary work in Natural Language Processing such as Transformers and language models have advanced the text summarization task.

In our project we utilized the HuggingFace Pytorch transformers library to run extractive summarizations for any given web scraped news article. The model we used is the Bert-Extractive-Summarizer.

3.2.3.1 Dataset

The CNN / DailyMail Dataset [3]is an English-language dataset containing just over 300k unique news articles as written by journalists at CNN and the Daily Mail. The current version supports both extractive and abstractive summarization, though the original version was created for machine reading and comprehension and abstractive question answering.

The dataset is uploaded on Kaggle and was used to measure the ROUGE score. The dataset consists of news articles and highlights, in two separate columns. The CNN articles were written between April 2007 and April 2015. The Daily Mail articles were written between June 2010 and April 2015. Below is a snippet of the dataset

CNN-DailyMail News Text Summarization

[3] https://www.kaggle.com/datasets/gowrishankarp/newspaper-text-summarization-cnn-dailymail

id	article	highlights
		Experts question if packed out planes are putting passengers at risk . U.S consumer advisory group says minimum space must be stipulated .
92c514c9	Ever noticed how plane seats appear to be getting smaller and sm	Safety tests conducted on planes with more leg room than airlines offer .
2003841c	A drunk teenage boy had to be rescued by security after jumping i	Drunk teenage boy climbed into lion enclosure at zoo in west India .
91b7d231	Dougie Freedman is on the verge of agreeing a new two-year deal	Nottingham Forest are close to extending Dougie Freedman's contract .

Table 4: shows the dataset used in text summarization

Feature	Mean Token Count
Article	781
Highlights	56

Figure 11:shows the average token count for the articles and the highlights

The data was split into 3 datasets, train, validation and test. Since we needed the data for testing, we only used the test dataset.

Dataset Split	Number of Instances in Split
Train	287,113
Validation	13,368
Test	11,490

Figure 12:The data was split into 3 datasets

3.2.3.2 Model Selection

Extractive text summarization with BERT

Extractive text summarization requires the model to 'understand' the entire text first then it picks out the right keywords and gathers these keywords to draw a conclusion or make sense of it. Moreover, it needs to not lose any information either. BERT does all of this with great speed and accuracy, and it was chosen because of its superior performance to other NLP algorithms on sentence embedding.

The BERT summarizer works in two steps, first it masks out 10% or 15% of the words and then attempts to predict those words. The other step is that it takes an input sentence, and a candidate sentence then predicts whether the candidate sentence properly follows the input sentence.

The BERT model has 340 million parameters which makes it large enough to generate reliable summaries. The core BERT implementation utilized the pytorch-pretrained-BERT library from HuggingFace.

3.2.3.3 Results

The performance of the text summarizer was evaluated using just the first 100 articles from the dataset. The evaluation metrics used to gauge the summarizer's effectiveness include precision, recall, and the F1 score.

Calculation of Metrics

The precision, recall, and F1 score were calculated based on the following formulas, as implemented in the code:

- **Precision**: The ratio of true positive (TP) tokens (tokens present in both the generated summary and the reference summary) to the total number of tokens in the generated summary. This is expressed as:

$$Precision = \frac{TP}{TP + FP}$$

Equation 2 Precision Formula

where FP denotes false positive tokens (tokens present in the generated summary but not in the reference summary).

- **Recall**: The ratio of true positive (TP) tokens to the total number of tokens in the reference summary. This is given by:

$$Recall = \frac{TP}{TP + FN}$$

Equation 3 Recall formula

where FN represents false negative tokens (tokens present in the reference summary but not in the generated summary).

- **F1 Score**: The harmonic mean of precision and recall, providing a single metric that balances both concerns. It is calculated as:

$$F_1 = 2 \cdot \frac{\mathrm{precision} \cdot \mathrm{recall}}{\mathrm{precision} + \mathrm{recall}} = \frac{\mathrm{TP}}{\mathrm{TP} + \frac{1}{2}(\mathrm{FP} + \mathrm{FN})}$$

Equation 4 F1 score formula

The average results of the model were:

- Precision: 0.3086
- Recall: 0.7708
- F1-score: 0.4191

The results show a moderate performance of the text summarizer because the precision value suggests that on average only about 31.83% of the tokens in the generated summaries are present in the reference summaries. Although this is an extractive method, still BERT can generate sentences that might not use the same tokens but can still mean the same. Furthermore, the higher average recall of 0.6020 indicates that the summarizer successfully captures around 60.20% of the relevant tokens from the reference summaries. However, the F1 score, which balances precision and recall, averages to 0.4108. This suggests that there is room for improvement in achieving a better balance between precision and recall.

3.3 Environment

We have used Google Colab, also known as “Colaboratory”, which is a cloud-based Jupyter notebook environment provided by Google Research and also we have used anaconda’s jupyterlab environment

Software Environment

The pre-processing and training of the models were done using two different tools, in Google Colaboratory graphic processing environment (GPU) which is a cloud based Jupyter notebook environment where we used the NVIDIA Tesla T4 with 15 GB of Ram in our Colab’s environment. Three of our 4 models were pre-processed and trained in Google Colaboratory graphic processing environment. The second environment was using JupyterLab as specified below in the figure.

Technology/tools	Description
Windows 10	Used operating system
JupyterLab	web-based interactive development environment for notebooks, code, and data.
Visual studio code	Open-source code editor strong support for a wide range of programming
Google Colab	Cloud-based jubeter noebook used for data analysis , machine learning and other computational tasks

Table 5:Shows Environment Software

Hardware Environment

For specific parts of the project, we have used the following hardware environment to develop and train some of the models.

Environment	Specifications
RAM	16.0 GB
Processor	AMD Ryzen 7 5800H with Radeon Graphics
Graphical card	Nvidia GeForce RTX 3070
Hard Disk	954 GB SSD
Screen	15.6

Table 6:Shows Environment hardware

Programming Language and Libraries

Python serves as the primary programming language for implementing and developing our project. It is an interpreted, high-level, and versatile programming language that supports multiple programming paradigms. Python was instrumental in training and testing machine learning and deep learning algorithms. We utilized several key libraries and frameworks throughout our work, including Pandas for data manipulation and analysis, NumPy for numerical computations, Matplotlib for data visualization, TensorFlow for implementing neural networks and deep learning models, and Keras for simplifying model construction and experimentation. These tools collectively supported various stages of our project, including preprocessing, training, testing, evaluation, and analysis of results, ensuring efficient development and comprehensive exploration of machine learning methodologies.

3.4 Chatbot

This section discusses the implementation of the NAS-AI Chatbot in detail. NAS-AI is a telegram Chatbot built using Googles' Dialog Flow, Flask framework for python, Web Scraping using BeautifulSoup and Ngrok, a cross-platform application. The chatbot interface will allow users to interact with the system. The figure below shows the detailed architecture of the NAS-AI chatbot.

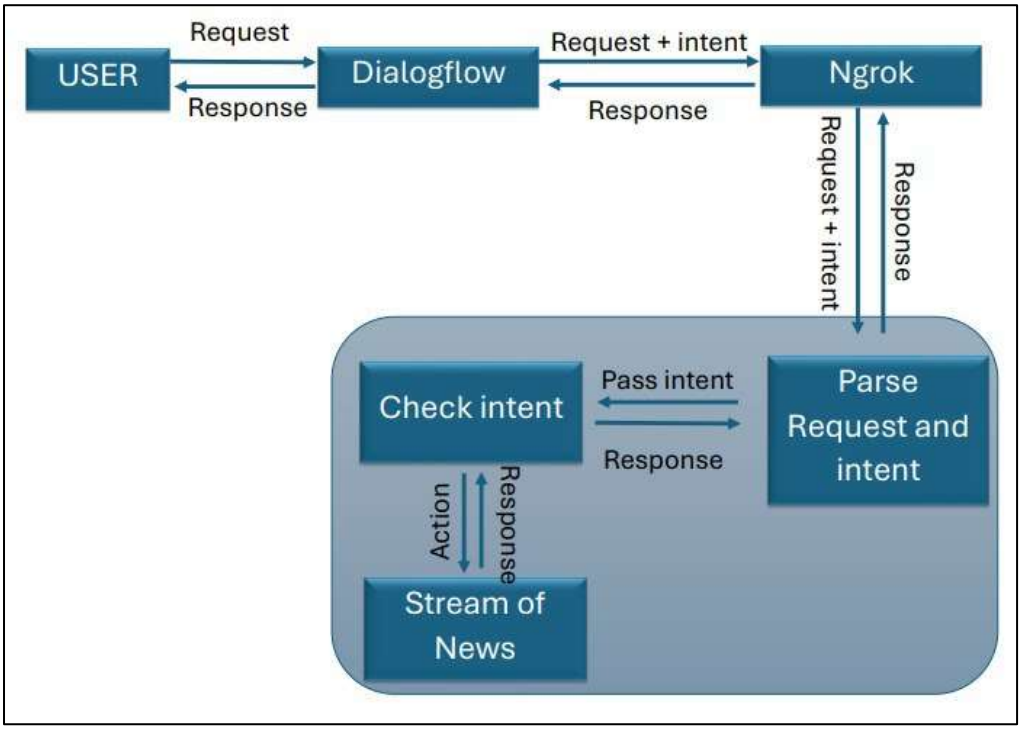


Figure 13 Shows an overview of the connections

We have implemented the models, as previously mentioned, on Google Colaboratory. Each model is pre-processed and used in a distinguished Jupyter Notebook resulting in 4 different models in 2 different Jupyter Notebooks. Our goal was to connect the user's query to all 4 models so that it can be processed, and the user can use the models. The connection was done using three important tools: Dialog Flow, Flask and Ngrok. When a user issues a command to the chatbot, it activates the models and begins producing a report of the classification of each article collected by the web scraper, which is displayed as a stream of news stories, with each individual piece being referred to as an article item. Each article item will have an article as well as a report. The article report describes a single article as Breaking news or not, specifies its topic, generates a descriptive headline and an informative summary for the article.

3.4.1 Dialogflow

Dialogflow, developed by Google, is a comprehensive platform for creating conversational interfaces such as chatbots and voice applications. It utilizes advanced Natural Language Understanding (NLU) to interpret user inputs and extract meaningful information, allowing developers to define user intentions (intents) and relevant data

points (entities). Dialogflow supports multiple languages, making it suitable for global applications. It provides a user-friendly interface for designing conversations, integrating with various messaging platforms, and leveraging machine learning to improve interactions over time.

In Dialogflow, key concepts such as intents, entities, contexts, fulfillment, training phrases, and responses work together to create intelligent and context-aware chatbots capable of understanding and responding to user inputs naturally and effectively.

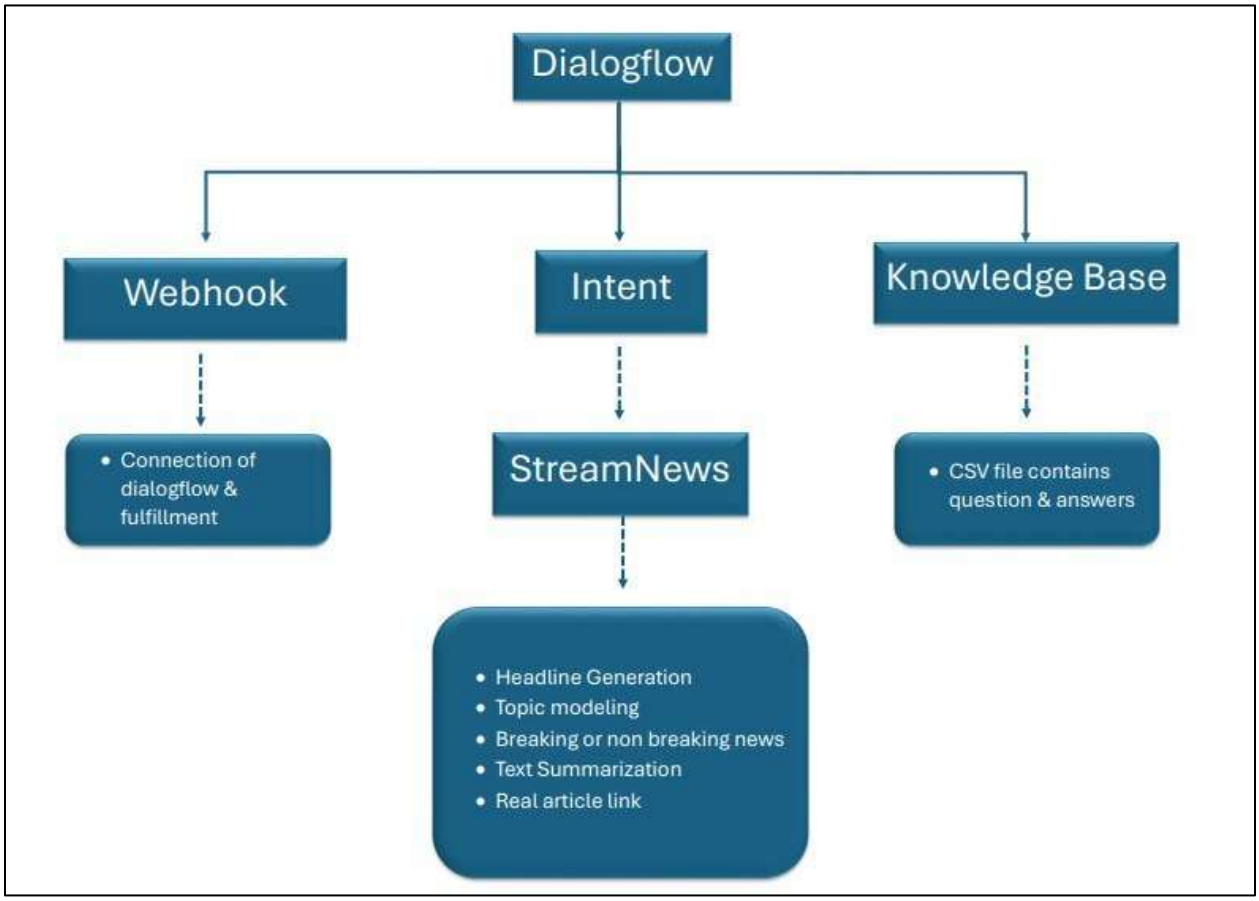


Figure 14:shows dialogflow usage

3.4.1.1 Intents

Intents define the purpose of a user's input, allowing the chatbot to understand what the user wants to achieve. In our case, we had three main intents:

- The Default Welcome Intent

- The Default Fallback Intent
- The StreamNews Intent

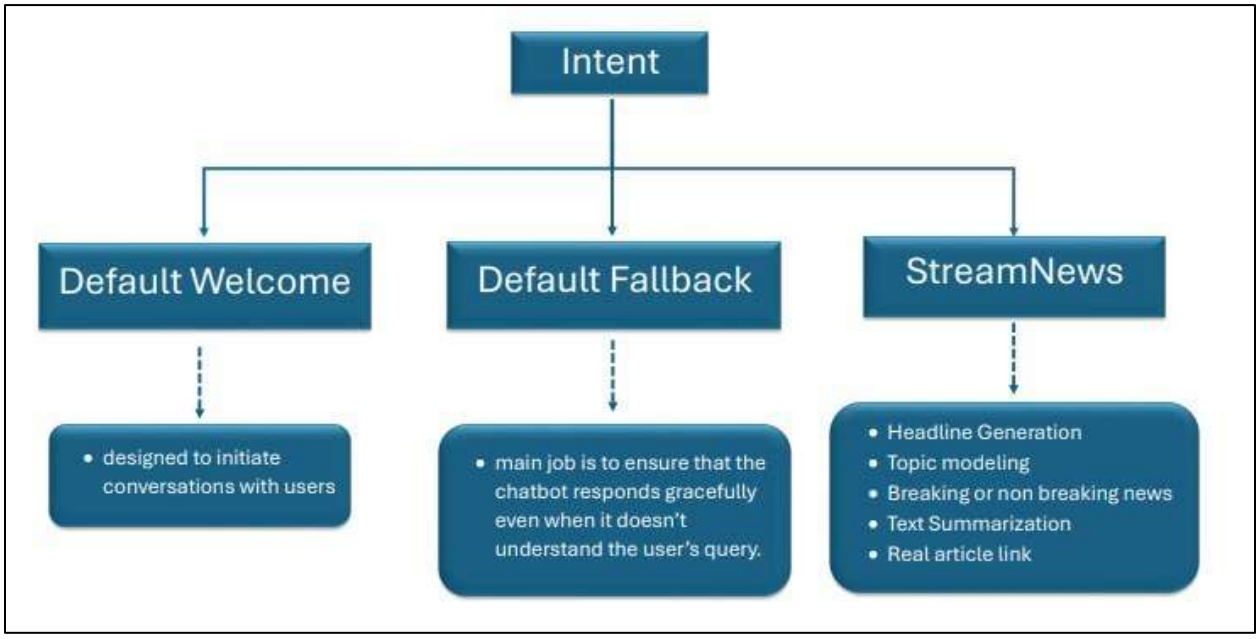


Figure 15:shows intents used in project

The default welcome intent is a pre-built intent in Dialogflow that is designed to initiate conversations with users. It gets triggered when a user first interacts with the chatbot, it is used typically to greet the user and set the stage for the interaction. The default welcome intent includes predefined training phrases such as 'hello', 'hi', 'hey there' and 'good morning' which help the chatbot identify common greetings.

The default fallback intent in Dialogflow is also pre-built and pre-configured to handle situations where the chatbot cannot match a user's input to any of the defined intents. This intent's main job is to ensure that the chatbot responds gracefully even when it doesn't understand the user's query. The default fallback Intent includes generic responses like 'I don't understand that' or 'Can you please rephrase?' which directs the user to ask their question in a different way. This intent is of utmost importance as it helps maintain the flow of conversation.

The StreamNews Intent is a custom-made intent we have designed to handle the user's request/input as a way of asking for the stream of daily news. The StreamNews Intent is

the core of the chatbot as it is responsible for identifying the request for news and triggering the models and the rest of the backend.

3.4.2 The Fulfillment Logic

Fulfillment in Dialogflow is a feature that allows the chatbot to provide dynamic custom responses and perform backend actions by interacting with an external service enabling the chatbot to go beyond the static replies and handle complex tasks and access real-time data making the conversations more interactive and useful. Fulfillment involves setting up webhooks that Dialogflow calls when an intent is triggered. This webhook can connect to a server or cloud function that processes the request and sends back a response.

3.4.3 The Flask Server

In our project the fulfillment allows the chatbot to interact with a Flask server that includes APIs for fulfilling Dialogflow requests. Flask is a lightweight and flexible web framework for Python that is often used to build web applications, including APIs for fulfilling Dialogflow requests. We have implemented in the Flask servers' script a webhook endpoint to handle requests from Dialogflow, process data and respond appropriately. We have created a Flask app with a 'webhook' endpoint to handle POST requests from Dialogflow.

3.4.4 The Webhook

Webhooks in Dialogflow allow chatbots to interact with external services by sending user queries to a specified URL, enabling dynamic and contextually relevant responses through custom backend logic or what we call the fulfillment logic. The StreamNews intent has webhook enabled to allow the chatbot to access external services or databases.

By enabling the webhook, Dialogflow can send a request to a particularized URL, the flask app (fulfillment logic), containing the details about the user's query and the current conversation context. On the other hand, the flask app processes this request,

performs necessary actions and then returns a response to Dialogflow, the response is then used to formulate a precise and informative reply to the user.
Inside the flask app (fulfillment logic) we have configured the webhook URL by specifying the URL of the webhook endpoint where Dialogflow will send the POST requests.

For the webhook to reach the flask app, which resides in the localhost, there should be a way to connect the flask app, chatbot and the models all together. Ngrok has been proven to be an extremely helpful tool in Localhost tunneling.

3.4.5 Ngrok

Ngrok is a cross-platform application that helps in exposing local web servers to the internet. It runs a local web server on its own sub-domain and connects the local development box to the internet using Tunnelling. We chose Ngrok because it is an excellent tool for testing local applications as if they were running on a live public server. Public hosting of an application requires a monthly subscription which was not suitable for our prototype, therefore, Ngrok was a very suitable choice as it offers secure tunnels to our local host without charge enabling our local host to be on the internet. It provides a temporary public URL that can be used to receive requests from Dialogflow's webhook and enable real-time interaction with the local server without needing to deploy it on a live server.

3.4.6 Connections

The main goal was to connect the chatbot with the models enabling the chatbot to provide a dynamic stream of news articles, each with their distinguished headline, summary, topic and a label if it's breaking news.

The first step was to create the fulfillment logic as a flask app which included all the models loaded, ready to be used, and the webhook service that will handle all the incoming requests from Dialogflow, process the data and perform the necessary actions then return a response in the required format.

The Fulfillment logic script (flask app) was implemented on google colab. The four models, as mentioned previously, are a topic modeling model, a BERT based breaking news classifier, headline generator and a BERT summarizer. The topic modeling and the BERT breaking news classifier each are pre-processed and trained in 2 different jupyter notebooks therefore, we have saved them as pickles and loaded these pickles in the fulfillment logic script. The headline generator and the BERT summarizer are both implemented in the fulfillment logic script.

3.4.7 The Web Scraper

The core idea of our project involves scraping the latest news, for which we selected CNN-Lite due to its diverse range of topics and daily updates. To achieve this, we employed the Python web scraping library BeautifulSoup to parse HTML and XML files, alongside the popular HTTP request library, requests. After importing these libraries, we utilized the website's inspect function to identify the exact web page elements needed for extraction from the CSS. We then developed a function to automate this process, limiting the scraping to 30 articles. The extracted links were intended for streaming, while the articles themselves served as input for models to generate summaries, headlines, classifications (breaking/non-breaking), and topic categorizations.

Implementing

In the beginning of the fulfillment logic, the models are loaded then the web scraper scrapes 20 articles. Next, the headline generator model is implemented and utilized to generate headlines for each of the 30 articles, then returns a list of the headlines. The BERT summarizer is then used to generate 30 summaries to each article and store them for the webhook to later on display them to the user.

The core of the fulfillment logic is to set up a Flask web server with a webhook endpoint, then expose the server to the internet using ngrok. It should also integrate the server with the chatbot through Dialogflow to process specific intents.

After loading the models and implementing the headline generator and text summarizer, it was time to set up the Ngrok secure tunnel. We specified a port number

on which the Flask app will run locally and started the ngrok tunnel to get the public URL which will be used in Dialogflow's webhook. The Flask-based webhook is designed to handle and fulfill requests from a Dialogflow chatbot.

Next was the Flask Application setup where we created a flask application instance to create the webhook Endpoint in it. We define a route ('/webhook') that listens for POST requests. Dialogflow sends users queries to the endpoint for processing.

The webhook service specifies a flask route to handle POST requests from Dialogflow which sends queries to the endpoint for processing. The requests are in a JSON format and include important detailed information about the user's query, the detected intent, parameters, contexts, and other relevant metadata that helps process user inputs and generate appropriate responses.

If the function detects the StreamNews intent, the webhook function handles it by processing a list of news articles, classifying each article and generating a response. The function loops through a list of scraped articles and classifies each article using the 'breaking_news_classify' function, which uses the BERT classifier model to classify articles as breaking or non-breaking news. and the 'topic_classify' function which uses the topic model to identify the topic of an article based on similarity scores.

Each response includes a link, classification, topic, headline, and summarization of the article. The loop stops after processing 5 articles to prevent overwhelming the user with too much information. The responses are formatted into a JSON structure suitable for Dialogflow's fulfillmentMessages. The system also handles unknown intents by sending a default response.

4.4.8 Knowledge Base

The knowledge base feature in Dialogflow enables the integration of pre-existing information into the chatbot, allowing for efficient and accurate responses to user queries. By leveraging documents, FAQs, and structured data sources, the knowledge base can automatically generate answers without the need for extensive training. This

feature enhances the chatbot's ability to provide quick and consistent information, improving user experience and reducing manual effort in managing content updates. We have incorporated the knowledge base feature to enhance the user experience by providing immediate and relevant information. This was achieved by creating a knowledge base and adding a document titled "Q&A," which is a CSV file containing a comprehensive list of questions and answers. The questions follow the format "who is ____," where the blank is filled with the name of a famous person from various fields. For example, one of the questions is "who is Joe Biden," pertaining to the politics domain. Below is a figure showcasing the format of the dataset:

Who is Angela Merkel?	Angela Merkel is a German politician who served as Chancellor of Germany from 2005 to 2021, known for her pra
Who is Nelson Mandela?	Nelson Mandela was a South African anti-apartheid revolutionary and political leader who served as President of
Who is Winston Churchill?	Winston Churchill was the Prime Minister of the United Kingdom during World War II and again from 1951 to 1955.

Table 7:shows CSV file used in Knowledge base

The Q&A document includes 205 questions about notable individuals across several fields such as entertainment, sports, and politics. This feature was strategically added to the chatbot to support users while they read news articles, anticipating the types of questions they might ask. By providing quick access to information about influential figures, the knowledge base helps users gain a deeper understanding of the news content they are engaging with. The answers provide the user with quick, short and comprehensive information for questions that may come into the user's mind.

All in all, the chatbot uses Dialog flow, Flask, Ngrok and webhooks technologies collectively to handle user requests and respond with dynamic custom responses that utilize the deep learning models we have previously implemented so that the chatbot fully functions as a facilitated way of news consumption to users.

Chapter 4: Conclusion

4.1 Future work

Our chatbot's core benefit to consumers is its capacity to provide a streamlined, tailored, and trustworthy news consumption experience; but there are numerous additional functions we may add to it.

We aim to continue working on our system and add the following features. Firstly, the user would be able to give the chatbot an article, and given this article, the chatbot generates a headline to it.

Secondly, to make the experience even more personalized, the chatbot will ask the user about their interests and from those interests, the chatbot will be able to extract information about the user preferences in news like in which topic (field) they like to read so it will stream only the news from their preferred topic. This will be implemented with GPT technology and more advanced training and fine-tuning.

Thirdly, the chatbot will be able to detect hate speech from articles scrapped from social media networks and classify the degree of hate speech and what kind, if it's offensive neutral or direct hate speech. The chatbot can also classify the sentiment within each article, whether it's bad, good or neutral.

Moreover, the chatbot can, as we mentioned above, take an article from the user and this will be useful in so many other functionalities to be presented to the user but firstly, the system will have to detect if the post/tweet is a piece of news in order to be able to drive useful insights from it, so the system will also have a news classification feature.

Chatbot gives users the option to choose which type of articles they want to read and the system streams only the articles that are relevant to the user.

Most importantly, the feature we really want to implement is for the models to be able to understand the Arabic language and stream and process Arabic articles and generate Arabic headlines and summaries.

4.2 Challenges

The need for high-performance GPUs, extended training times, and the costs associated with hardware can be formidable barriers. Addressing those barriers requires a lot of financial resources and hardware resources we did not have.

The implementation of breaking and non-breaking news classification represented a substantial challenge due to the novelty of the classification and the absence of pre-existing data. Creating a custom dataset required significant time and effort, involving horrendous data collection, manual annotation, and rigorous quality assurance processes.

Testing the Llama2 generation model revealed significant challenges due to the limitations of traditional NLP evaluation metrics like BLEU. These metrics, focusing on n-gram token length rather than the meaning of the text, required us to use human review methods.

4.3 Conclusion

In conclusion the project's primary goal is to enhance user experience by developing a system that delivers concise, accurate, and well-categorized news through advanced technologies like Natural Language Processing (NLP) and topic modeling, ensuring users can efficiently access and discern relevant information amidst the overwhelming influx of news content.

We developed a system to streamline the news consumption process, providing a seamless and smooth user experience. Using a chatbot interface, the system delivers analyzed and summarized daily news from trusted sources. The chatbot can generate headlines and summaries for news articles, classifying them as breaking news or otherwise, and indicating their genre. This approach ensures users receive essential information quickly, mitigating issues of information overload, disorganization, and unreliability. The chatbot is integrated with Telegram, facilitating user interaction through analyzed news batches. The project is divided into two main parts: chatbot creation and model creation/finetuning. Chatbot creation involves using tools like DialogFlow, Flask, and ngrok to interface user queries with reliable responses. The model creation part leverages large language models (LLMs) and various deep learning techniques.

The project includes topic modeling to group similar articles, breaking news classification, headline generation, and text summarization. A chatbot, developed with Google Dialog Flow and integrated via Flask and Ngrok, serves as the user interface, delivering labeled, summarized news articles. Web scraping ensures the system provides up-to-date news by continuously feeding new data into the models.

Overall, the News Analysis System represents a significant step toward combating misinformation, organizing news content, and providing users with a personalized and reliable news consumption experience in the digital age.

Appendix A

A demo of the Chatbot

This appendix consists of multiple screenshots showcasing NAS-AI's, the chatbot, results and performance on telegram and a real full conversation between the user and the chatbot.



Figure 16 showing the logo of the chatbot

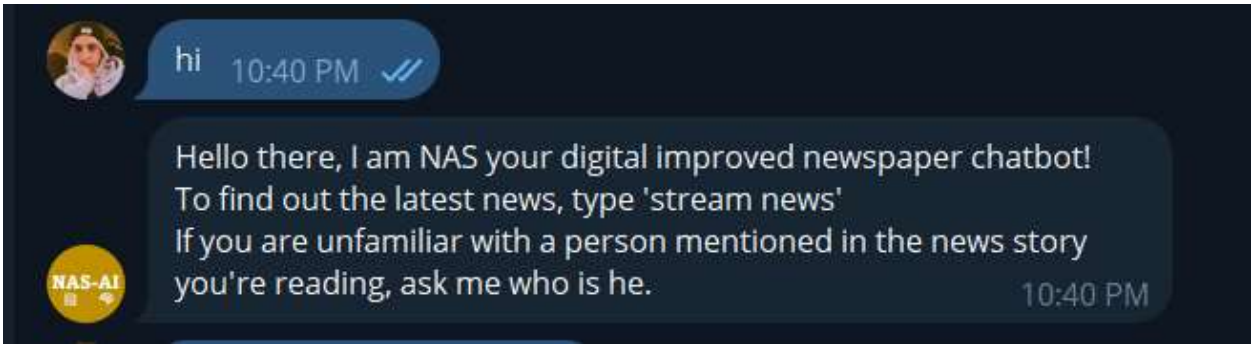


Figure 17 showing a scenario where a user greets the chatbot



Figure 18 one article and its report

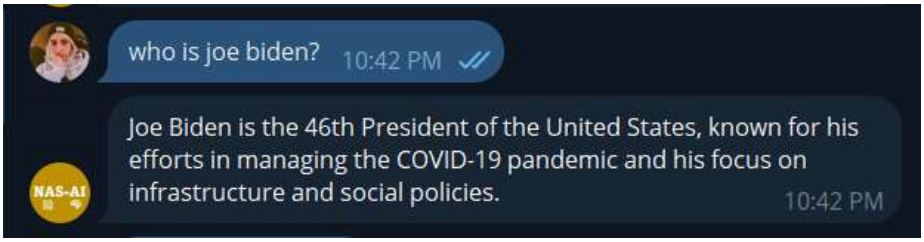


Figure 19 A question and the answer

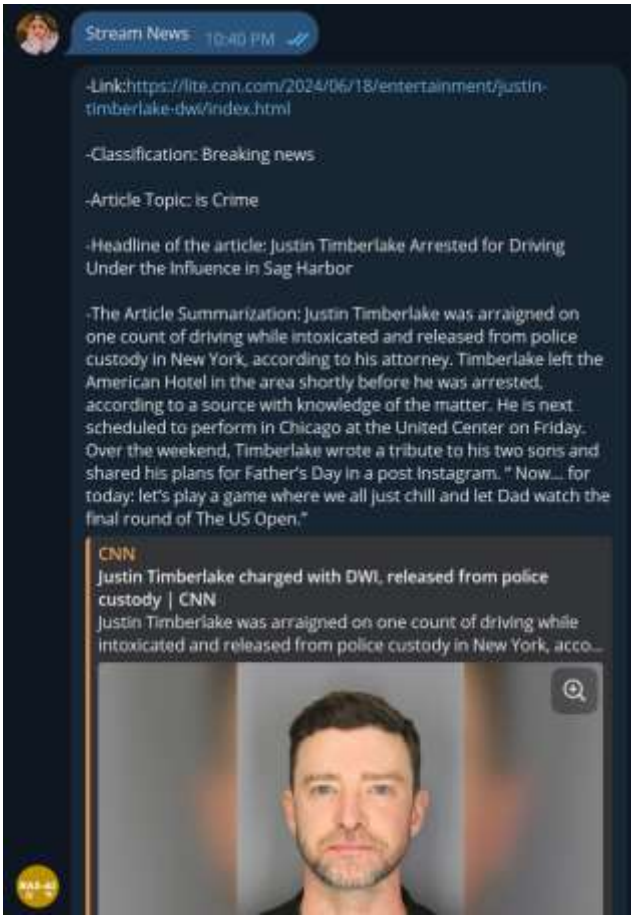


Figure 20 shows the user requesting news

The Llama2 prompt template

```
# System prompt describes information given to all conversations
system_prompt = """
<s>[INST] <<SYS>>
You are a helpful, respectful and honest assistant for labeling topics.
<</SYS>>
"""
```

Figure 21: shows system prompt in topic modeling



Figure 22: shows User prompt in topic modeling



Figure 23: shows main prompt in topic modeling

We then concatenated all the prompts into one single prompt to be fed into the llama2 model.

References

[1] M. Erraki, M. Youssfi, A. Daaif, and O. Bouattane, "NLP Summarization: Abstractive Neural Headline Generation Over A News Articles Corpus," in *Proc. International Conference on Data Science (ICDS 2020)*, Oct. 2020, doi: 10.1109/ICDS50568.2020.9268776.

[2] H. Batra, G. K. Palawat, K. Gupta, Priadarshana, Supragya, D. Bajaj, and U. Bharti, "CovFakeBot: A machine learning based chatbot using ensemble learning technique for COVID-19 fake news detection," *Int. J. Artif. Intell. Soft Comput.*, vol. 7, no. 3, pp. 228-241, 2022. doi: 10.1504/ijaisc.2022.126344.[2]

[3] H. Q. Abonizio, A. P. A. C. Barbon, R. Rodrigues, M. Santos, V. Martínez-Vizcaíno, A. E. Mesas, and S. Barbon Junior, "How people interact with a chatbot against disinformation and fake news in COVID-19 in Brazil: The CoronaAI case," *International Journal of Medical Informatics,* vol. 177, p. 105134, 2023, https://doi.org/10.1016/j.ijmedinf.2023.105134.

[4] [Article 1] X. He et al., "Compressed-state compression of deep neural networks for edge intelligence," *Future Generation Computer Systems*, vol. 145, pp. 387-400, 2023. doi: 10.1016/j.future.2023.06.014.[Article 2] Z. Li, J. Wu, J. Miao, and X. Yu, "News headline generation based on improved decoder from transformer," *Scientific Reports*, vol. 12, p. 11648, 2022. doi: 10.1038/s41598-022-15817-z.

[5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLORA: Efficient Finetuning of Quantized LLMs," *arXiv preprint arXiv:2305.14314v1*, May 2023. [Online]. Available: https://arxiv.org/abs/2305.14314v1 [5]

[6] M. Grootendorst, "BERTopic: Neural topic modeling with a class-based TF-IDF procedure," *arXiv preprint arXiv:2203.05794v1*, Mar. 2022. [Online]. Available: https://arxiv.org/abs/2203.05794v1 [6]

[7] E. Levas, D. de Leon, and Y. Li, "A Smart Class Chatbot for Improving Student Learning and Engagement," in *Proc. 2021 Computer Science Conference for CSU Undergraduates*, Virtual, Mar. 2021. [Online]. Available: https://scholarworks.calstate.edu/

[8] D. Miller, "Leveraging BERT for extractive text summarization on lectures," arXiv, Jun. 2019. doi: 10.48550/arXiv.1906.04165.[8]

[9] Meta, "Llama 2 7B Chat Model," Hugging Face, Jun. 2023. [Online]. Available: https://huggingface.co/meta-llama/Llama-2-7b-chat-hf. [Accessed: Jun. 22, 2024].[9]

[10] D. Deychakiwsky, "Open Foundation AI Models - Meta's Llama2-7B-Chat," Deylemma, 2023. [Online]. Available: https://daniel-deychakiwsky.github.io/posts/open-foundation-ai-models-metas-llama2-7b-chat/. [Accessed: Jun. 22, 2024].

[11] Meta, "Llama 2: Open Foundation AI Models," Hugging Face, 2023. [Online]. Available: https://huggingface.co/meta-llama/Llama-2-7b-chat-hf. [Accessed: Jun. 22, 2024].

[12] "Making LLMs even more accessible with bitsandbytes, 4-bit quantization and QLoRA," Hugging Face, 2023. [Online]. Available: https://huggingface.co/blog/4bit-quantization. [Accessed: Jun. 22, 2024].

[13]. Tran, "Accuracy, Precision, and Recall in Deep Learning," Paperspace Blog, Jun. 2023. [Online]. Available: https://blog.paperspace.com/accuracy-precision-recall/. Accessed: Jun. 22, 2024.